Smishing Detection Beyond English Benchmarks: A Critical Review of Code-Mixed Indic Text and On-Device Language Models

SIDHARDH S[1], ADVAITH KAMATH[1], JOEL G. BERT[1], T. AMARSAINADH[1], BILEESH P. BABU[1], and GOKUL YENDURI[1]

[1]School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, Andhra Pradesh 522237, India (e-mail: sidhardh215@gmail.com; advaithkamath25@gmail.com; joelgbert15@gmail.com; amar.22mis7013@gmail.com; bileeshbabu.p@vitap.ac.in; gokul.yenduri@vitap.ac.in)

Corresponding author: Bileesh P. Babu (e-mail: bileeshbabu.p@vitap.ac.in).

This work did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

ABSTRACT Smishing detection is often reported as nearly solved on English SMS benchmarks, yet a deployable detector must also cope with code-mixing, Romanized Indic text, adversarial rewriting, privacy constraints, and limited handset resources. This critical integrative review examines those requirements together. The evidence base contains 169 records selected through 12 July 2026, with metadata verified through 28 July 2026, spanning direct SMS detection, code-mixed Indic processing, language-model-based threat detection, and resource-efficient inference. An eligibility-coded comparison ledger contains 48 direct SMS studies: 36 are English-only, eleven include a known non-English or multilingual evaluation, seven report an explanation mechanism, four evaluate robustness, and one reports both time and processing memory on a named handset. We add a preliminary validation on open tri-class English and Bangla corpora. Conflict quarantine and exact deduplication reduce 5,971 English rows to 5,797 independent-label records and 2,772 Bangla rows to 1,204. Across five fixed splits, grouping character-trigram near-duplicate components lowers mean macro-F1 relative to naive splitting by 0.031 for a word SVM and 0.027 for a character SVM. A provenance audit finds that 4,751 of 5,159 unique UCI SMS messages exactly or nearly overlap the English source, leaving only 408 contamination-clean transfer cases. These findings demonstrate protocol executability and quantify dataset dependence without estimating code-mixed Indian smishing performance. The review contributes an evidence-layered taxonomy, an empirical dataset audit, a minimum reporting checklist, and a governance-aware benchmark design for tri-class, dual-script Malayalam–English detection.

INDEX TERMS Code-mixed text, edge artificial intelligence, Indic natural language processing, low-resource languages, mobile security, on-device inference, smishing, SMS phishing.

I. INTRODUCTION

SMISHING is phishing delivered through SMS. A typical lure presents an urgent pretext and asks the recipient to open a link, call a number, reply, disclose a credential, or transfer money [1], [2]. Detection is difficult for a simple reason: the message must be judged from very little text, often before any reputation service has seen the sender or destination. U.S. Federal Trade Commission data report $470 million in 2024 losses from scams that began with text messages [3]. This is a complaint-based text-scam statistic, not a complete or smishing-only loss estimate.

People are not reliable filters. In a controlled study of 187 participants, only 67.1% of smishing messages were identified correctly, while just 43.6% of legitimate messages were accepted as genuine [4]. False reassurance and false alarms are therefore both practical concerns. Measurements of live campaigns show shared infrastructure, high-volume delivery, and rapidly changing lures [5], [6]. Commercial tools also miss modern attacks [7], while generative models make fluent and personalized lures cheaper to produce [8], [9]. Smishing is consequently a joint problem in language understanding, adversarial security, human factors, and systems engineering.

India makes the language and deployment constraints especially visible. National cyber-fraud figures show a large overall burden, although they aggregate complaint channels and must not be interpreted as SMS-specific loss [10]. The Telecom Regulatory Authority of India has separately pressed operators to use artificial intelligence and machine learning against unwanted commercial communication and fraud [11]. A useful detector must therefore work in the form in which messages actually arrive, without turning a broad national fraud statistic into an unsupported claim about smishing.

Indian digital text frequently combines English with Malayalam, Tamil, Telugu, Kannada, Hindi, or other languages, and often writes the Indic language in Latin script [12], [13]. Spelling is variable, abbreviations are common, and language alternation can occur within a sentence or word. Standard multilingual encoders perform less reliably on transliterated Hindi and Malayalam than on native-script input [14]. In contrast, most SMS-filtering experiments still use English corpora derived from a benchmark released in 2011 [15]. The central mismatch is therefore broader than ''low resource'': training data, language representation, adversarial drift, privacy, and device limits all differ between the benchmark and the deployment setting.

The technical literature has moved through recognizable stages. Rules and sender heuristics came first [16], [17]. The SMS Spam Collection then enabled a long period of classical machine learning with lexical and engineered features [15], [18], [19]. Convolutional and recurrent networks learned representations directly from messages [20], [21]; pretrained encoders later improved transfer and data efficiency [22], [23]. Recent work uses language models as zero- or few-shot classifiers, generators for scarce-data augmentation, task-adapted compact models, and components of interactive systems [24]–[27]. Reported in-domain scores have risen, but the associated evidence often remains tied to old English data.

There are important counterexamples. Published work now covers Swahili smishing [28], Korean detection with generated explanations [29], and Bangla, Arabic–English, and Vietnamese SMS classification [21], [30], [31]. The deployment record is thinner. Cloud-hosted language models require message content to leave the device and add network and recurring inference costs [32], [33]. SpaLLM-Guard demonstrates that language models can detect SMS spam and can be tested under manipulation and drift, but it does not establish handset deployment [34]. Quantization and parameter-efficient adaptation can reduce the resource burden [35]–[37], and phone-oriented model families show that compact decoders are technically plausible [38]–[40]. General-purpose quantization results, however, do not establish security performance after compression [41]. COPS provides a valuable direct precedent by reporting time and memory on a named handset [42], and multilingual LoRA has been evaluated for English, Bengali, and Swahili SMS [43]. The 169-record evidence base contains no evaluation that combines code-mixed Indian-language smishing, explicit model precision, adversarial robustness, and device-level latency, memory, and energy.

Prior reviews do not answer that combined question. Smishing surveys are broad but largely language-agnostic [1], [44], [45]; reviews of LLMs in phishing are not centered on SMS or Indic text [9]; phishing-webpage and mobile-phishing reviews address different threat surfaces [46], [47]; and small-language-model surveys treat efficiency without a smishing application [48]. Table 1 compares these scopes. This article is intended to connect them without treating evidence from one channel or task as though it were direct evidence for another.

The review makes five contributions.

C1. It separates direct SMS detector evidence from adjacent transfer and context evidence, and documents the provenance limits of the historical search (Section III).

C2. It organizes detection methods by signal, model family, language handling, and deployment location, with companion taxonomies for code-mixed processing and efficiency (Sections IV, IX, and X).

C3. It applies explicit direct-detector eligibility rules to the quantitative ledger, separating email, webpage, offensive-language, attacker-generation, and campaign-monitoring studies from direct SMS detectors. The resulting counts are explicitly limited to the comparison tables.

C4. It audits dataset reuse, score comparability, explanation, robustness, and deployment reporting, then instantiates the proposed leakage controls on open English and Bangla tri-class corpora without constructing a misleading cross-paper leaderboard (Sections XI–XIII).

C5. It turns the remaining evidence gaps into a staged benchmark design covering corpus governance, dual-script evaluation, Indic encoders, compact language models, adversarial testing, and named-device measurement (Section XIV).

The remainder of the article follows that progression. Section II defines the task and technical background; Section III states the review design and its evidence boundaries; Section IV introduces the organizing taxonomy; Sections V–VIII examine four model generations; Sections IX and X review the two enabling literatures; Section XI examines datasets and protocols; Section XII provides a preliminary controlled validation; and Sections XIII–XV synthesize the evidence and propose a research agenda. Fig. 1 summarizes the scope.

II. BACKGROUND AND PRELIMINARIES

A. SPAM, PHISHING, AND SMISHING

Unsolicited SMS traffic is not homogeneous, and the literature increasingly insists on separating three classes that early work conflated. *Spam* denotes bulk, unsolicited but not necessarily malicious messaging, typically promotional in intent [15]. *Phishing* denotes deceptive communication whose objective is to elicit sensitive information or an unsafe action from the recipient. *Smishing* is phishing instantiated over SMS, characteristically combining a fabricated pretext, a persuasion trigger, and an action vector, most often an embedded URL, but also callback numbers or reply prompts [1], [2].

The distinction is operationally consequential. A published tri-class corpus and its archived dataset distinguish ham, spam, and smishing [49], [50]; later multi-class work illustrates the difficulty of the boundary [51]. Persuasion theory supplies the concepts of authority, scarcity, liking, and reciprocity [52]; smishing studies use related cues for classification and explanation [25], [53].

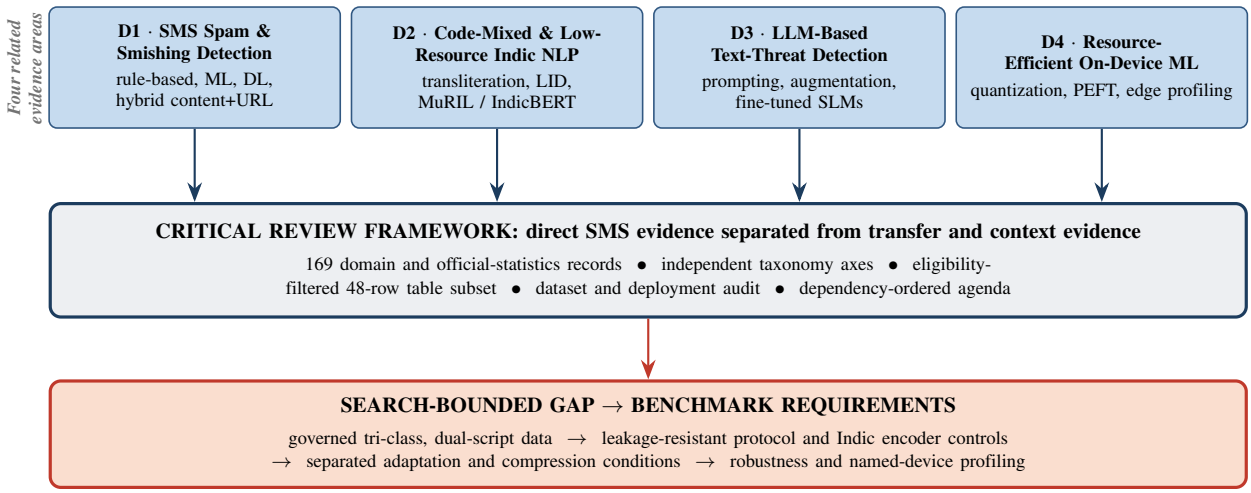

FIGURE 1. **Review scope and evidence flow. The review connects four areas while keeping direct SMS detector evidence separate from transfer and context evidence. ML, DL, NLP, and LID denote machine learning, deep learning, natural language processing, and language identification, respectively.**

TABLE 1. **Coverage of This Review Against Existing Secondary Studies. ✓ = Covered; ∼ = Partial; × = Not Covered**

Dimension	Goel & Jain [47] 2017	Al Saidat [45] 2024	Kulkarni [46] 2024	Van Nguyen [48] 2024	Rahaman [44] 2025	Pritom [1] 2026	LLM-phish [9] 2026	This review
Declared systematic/PRISMA protocol[a]	×	×	×	×	×	✓	✓	×
Smishing-specific focus	∼	✓	×	×	✓	✓	∼	✓
LLM-era detection methods	×	∼	∼	✓	∼	∼	✓	✓
Code-mixed / Romanized Indic text	×	×	×	×	×	×	×	✓
Indic encoders (MuRIL, IndicBERT)	×	×	×	×	×	×	×	✓
Quantization / 4-bit SLMs	×	×	×	✓	×	×	×	✓
On-device latency, memory, energy	×	×	×	∼	∼[b]	×	×	✓
Dataset and benchmark analysis	∼	∼	✓	×	✓	✓	∼	✓
Coded quantitative map[c]	×	×	×	×	×	×	×	✓
Gap-to-direction roadmap with priority ordering	∼	∼	∼	∼	∼	∼	∼	✓
Integrates all four literatures[d]	×	×	×	×	×	×	×	✓

[a]✓ only where the study declares a systematic/PRISMA protocol and the manuscript package supports that claim; this review is marked × because historical exports and a screening ledger were not supplied. [b]Federated learning is discussed, but no latency, memory, or energy figures are reported. [c]Quantitative mapping uses the 48 direct SMS rows shown in the comparison tables (Section XIII-A). [d]Smishing detection × code-mixed Indic NLP × LM-based text-threat detection × resource-efficient on-device inference.

Several properties make SMS a harder detection substrate than email. A single segment carries at most 160 GSM-7 characters, or 70 characters when UCS-2 encoding is used; longer messages are concatenated. This encourages abbreviation and irregular spelling. SMS also lacks the structured headers and attachment context available to email filters, while sender identifiers and domains can be rotated. Legitimate conversational language overlaps with the informal style an attacker can imitate [54], [55]. Live campaigns use homoglyphs, zero-width characters, URL shorteners, and frequent paraphrasing to degrade filters trained on static data [55], [56]. Recipients, meanwhile, often judge legitimacy from brand names, tone, and personal relevance—signals that an attacker can control [4], [57].

B. CODE-MIXING, ROMANIZATION, AND THE INDIAN SMS REGISTER

Code-mixing (or code-switching) alternates between two or more languages within a discourse, sentence, or word. In Indian digital communication it is often combined with *Romanization*: an Indic language is written in Latin rather than its native script [58]. The DravidianCodeMix corpus documents Tamil–English, Malayalam–English, and Kannada–English mixing, including intra-word switches, in naturally occurring online text [12]. Malayalam–English language-identification data likewise contain Malayalam, English, acronyms, and language-independent tokens within the same sentence [13]. These sources establish the form of the register, although neither is an SMS-usage prevalence study; the present review therefore avoids claiming that code-mixing dominates all Indian SMS traffic.

Three properties of this register are hostile to conventional pipelines. First, Romanized spelling is unstandardized: the same Hindi word may surface as ''jaldi,'' ''jldi,'' or ''jldii,'' defeating lexicon lookup and inflating vocabulary size [59]. Second, tokenizers trained mainly on native scripts or English fragment Romanized Indic words into uninformative pieces; controlled evaluations show systematic PLM deficits on transliterated Hindi and Malayalam [14]. Third, word-level language identification, a prerequisite for transliterate-then-classify pipelines, is itself non-trivial [13].

The resource ecosystem has responded. Dakshina supplies Romanization lexicons and parallel text [60]; Aksharantar provides 26 million attested pairs across 21 languages and trains IndicXlit, with Malayalam contributing the largest split at 4.1 million pairs [61]. MuRIL pre-trains on 16 Indian languages, English, and transliterated counterparts [62], while IndicBERT provides a compact ALBERT-style encoder over IndicCorp [63]. Together they form the natural normalization and encoder baselines for Indic short-text security.

C. FROM PRE-TRAINED LANGUAGE MODELS TO QUANTIZED ON-DEVICE LLMS

The transformer architecture [64] underpins two model families relevant here. Encoder pretrained language models (PLMs) in the BERT lineage [65] are commonly fine-tuned with a classification head. Multilingual variants such as XLM-R [66], MuRIL [62], and IndicBERT [63] extend this approach across languages. DistilBERT reports a smaller model and faster inference than its BERT teacher while retaining most of the measured language-understanding performance in its original evaluation [67]; that result does not establish the same trade-off for smishing. Decoder language models add instruction following and in-context classification [68]. Compact, open-weight families such as Phi-3, Gemma 2, and Qwen2.5 provide candidate backbones for constrained experiments [38]–[40], but their general benchmark scores should not be treated as SMS-security results.

Knowledge distillation transfers behavior from a larger teacher to a smaller student and is one route to model compression [69]. Post-training quantization (PTQ) instead reduces numerical precision after training. GPTQ uses layer-wise, approximate second-order information for weight-only quantization [35]; AWQ protects activation-salient weight channels [36]; and SmoothQuant and LLM.int8() address activation outliers in weight-and-activation settings [70], [71]. These methods have different calibration, kernel, and hardware requirements, so no method is a universal mobile default. Parameter-efficient fine-tuning (PEFT) is a separate operation: LoRA trains low-rank adapters over a frozen backbone [72], whereas QLoRA performs adapter training through a frozen four-bit NormalFloat backbone [37]. General evaluations show that four-bit PTQ can preserve much of the measured performance for some models and tasks, with material variation across methods and scales [41]; smishing accuracy at a stated deployed precision remains to be tested.

Mobile evaluation suites measure time to first token, token throughput, resident memory, and battery drain on real handsets [33]. Sustained-load studies further show that thermal behavior can change observed throughput [73]. No reviewed study reports accuracy, latency, memory, energy, and deployed precision together for a detector evaluated on adversarial, code-mixed Indic SMS. This is the joint measurement gap examined in the remainder of the review.

III. REVIEW DESIGN AND EVIDENCE BOUNDARIES

A. RESEARCH QUESTIONS

The review addresses five questions. **RQ1:** How have automated SMS spam and smishing detectors evolved from rules to language models, and which limitations persist across generations? **RQ2:** What evidence exists for code-mixed, Romanized, and low-resource Indic text, and how directly does it transfer to SMS security? **RQ3:** What is known about compression and on-device inference for privacy-sensitive text classification? **RQ4:** Which datasets and evaluation practices support credible comparison? **RQ5:** Which research tasks are prerequisites for a deployable code-mixed Indian-language detector?

B. SEARCH FRAMEWORK

The evidence search covered 1 January 2006 through 12 July 2026. Reference metadata and the publication status of named preprints were rechecked through 28 July 2026 without reopening the eligibility window. Sources were arXiv, IEEE Xplore, the ACM Digital Library, SpringerLink, Elsevier ScienceDirect, the ACL Anthology, Google Scholar for backward and forward citation chasing, and the IEEE DataPort, Mendeley Data, and TechRxiv repositories. Twenty-four query families covered SMS spam and smishing, code-mixed and Indic processing, language-model-based text-threat detection, efficiency and on-device inference, and adversarial robustness. The supplement records the canonical Boolean strings and the rules used to adapt them to each source.

Records were retained in the broader evidence base when they satisfied at least one of four conditions: they (i) proposed or evaluated automated SMS or short-text threat detection; (ii) contributed a directly transferable method for code-mixed, Romanized, or Indian-language classification; (iii) contributed language-model compression, parameter-efficient adaptation, or mobile inference evidence; or (iv) supplied a relevant dataset, evaluation method, or secondary synthesis. We excluded non-retrievable items, purely journalistic accounts, work with no text component, out-of-window publications, and superseded duplicates when an archival version was available. Peer-reviewed studies were preferred. Preprints were retained only when they supplied technically consequential evidence not yet available in an archival publication and are identified as preprints in the references.

The resulting scientific bibliography contains 169 records. That number is a cited evidence-base size, not a count of records identified, screened, or included in a systematic review. Historical database exports, exact query dates, deduplication decisions, and a record-level exclusion log were not

preserved. The present article is therefore presented as a critical integrative review; it does not claim PRISMA compliance or exhaustive coverage.

C. THREE EVIDENCE LAYERS

To prevent adjacent work from being counted as direct SMS evidence, the synthesis uses three layers. *Direct detector evidence* consists of original empirical work that reports automated, message-level SMS spam or smishing results. *Transfer evidence* covers email and webpage phishing, code-mixed and Indic classification, offensive-language tasks, and mobile language-model benchmarks that contribute a method but do not evaluate the target SMS task. *Context evidence* covers datasets without a detector result, surveys, human-subject studies, campaign measurement, attacker-side generation, standards, and foundational methods.

Tables 2–5 form a structured comparison map. A row is retained when it represents a distinct method family, language setting, evaluation property, or deployment result and reports enough detail for the table's fields. Applying the direct-detector rule to the 59-row comparison ledger excludes eleven adjacent records: two email PLM studies, seven email or webpage LLM studies, one attacker-generation study, and one campaign-monitoring study. Those rows remain visible and are marked as transfer or context evidence, but they are excluded from direct-detector counts. The resulting table-based set therefore contains 48 direct SMS studies: 18 classical, 13 deep-learning, nine PLM, and eight decoder-LLM entries. XLM-R with LoRA [43] is classified with PLMs because parameter-efficient adaptation does not turn an encoder into a decoder LLM.

The 48-row set is purposive rather than exhaustive: it maps the studies selected for the four generation tables, whereas other direct Indian-language detectors are discussed narratively in Section IX. All percentages reported from the ledger therefore describe these 48 tabled studies and are not literature-wide prevalence estimates.

D. CODING AND SYNTHESIS

For each row in the comparison ledger, we record publication year, primary methodological generation, communication channel, language scope, named corpus, task labels, headline metric, explanation mechanism, robustness evaluation, handset profiling, numerical precision, and energy reporting. A study can be discussed in more than one narrative section, but it receives one primary generation for plotting. The Bangla BERT–character-CNN hybrid [30] is assigned to the PLM generation. The supplement exposes each coded row together with its direct-detector eligibility decision so that every reported count can be regenerated.

The synthesis reports counts and missing-evidence patterns, but it does not pool accuracy or F1. Binary spam detection, tri-class smishing detection, private corpora, duplicate-contaminated random splits, cross-corpus tests, accuracy, macro-F1, and class-specific F1 are not exchangeable observations. Published percentages are therefore discussed only within each study's protocol, not as a leaderboard or a causal comparison of architectures.

E. TARGETED BOUNDARY CHECKS

Five targeted searches challenged the review's narrowest claims: (i) a code-mixed or Romanized Indian-language corpus distinguishing smishing from bulk spam; (ii) MuRIL or IndicBERT applied to Indian SMS threats; (iii) a decoder LLM evaluated on code-mixed Indian-language smishing; (iv) detection accuracy reported at an explicit quantized precision; and (v) latency, memory, and energy reported for a smishing detector on named hardware. These checks found important near misses. COPS reports handset time and processing memory [42]; KorSmishing Explainer evaluates Korean classification and rationales [29]; SpaLLM-Guard studies SMS LLMs under manipulation and concept drift [34]; and Mambina et al. provide Swahili smishing evidence [28]. The remaining gap is therefore the joint evaluation of code-mixed Indian-language smishing, suitable language models, adversarial robustness, and precision-aware device cost, not the universal absence of each component.

F. LIMITATIONS AND INTERPRETATION RULES

The review has four material limitations. First, the missing historical exports and screening log prevent reconstruction of a systematic record flow. Second, selection and coding were not documented as independent dual review, so reviewer judgment may have introduced selection or classification error. Third, fast-moving topics require some preprints whose metadata and conclusions may change. Fourth, reported results were not reproduced, and heterogeneous datasets and protocols preclude causal cross-study ranking. Negative findings apply to the listed sources and cut-off date, and near-miss evidence is named wherever it narrows a claim.

IV. A TAXONOMY OF SMISHING DETECTION APPROACHES

Fig. 3 presents the taxonomy used in Sections V–VIII. Its axes should be read independently. One records the evidence consumed by a detector, a second records the model family and role, and a third records where inference or training occurs. The familiar ''generations'' are an organizational device, not a claim that later methods replace earlier ones.

Non-content-based approaches use signals other than message text: sender blocklists, allowlists, and reputation; URL or domain reputation and resolution behavior; and campaign-level infrastructure. Attackers can rotate sending identities and domains faster than reputation lists propagate [5], [6]. Infrastructure analysis helps operators characterize and disrupt campaigns, but cannot by itself classify a first-seen lure in an individual inbox.

Content-based approaches classify the message text. *Rule and heuristic* systems encode patterns over keywords, URLs, and numbers [16], [17]; they are transparent but require maintenance. *Classical machine learning* learns from token n-grams, character statistics, and URL flags [15], [19],

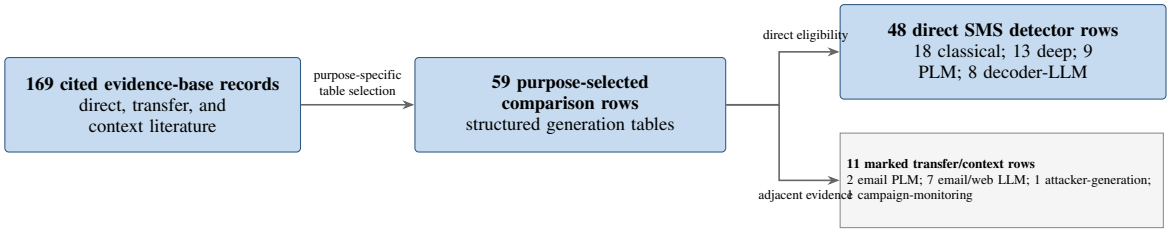

FIGURE 2. **Evidence structuring used for quantitative synthesis, not a PRISMA identification-and-screening flow. The 169-record cited evidence base is broader than the purposive comparison map. Eligibility coding of the 59 table rows retains 48 direct SMS detectors for quantitative counts and marks 11 rows as transfer or context evidence. Historical search-hit and exclusion counts are unavailable.**

[74]. *Deep learning* uses convolutional neural networks (CNNs), long short-term memory (LSTM), gated recurrent units (GRUs), and hybrids [20], [21], [75]. *Transformer and pretrained language model (PLM) fine-tuning* transfers bidirectional encoders and reports high in-domain scores on canonical benchmarks [22], [23], [30].

Large language model (LLM)-based work uses decoder models in several roles: zero- or few-shot classifier [24], [76]; data generator [25], [51]; task-adapted compact detector [77]; and component of an agentic or ensemble system [26], [78]. XLM-R with low-rank adaptation (LoRA) [43] is a PLM adaptation result, even though it is discussed with recent parameter-efficient work. Attacker-side generation and campaign analysis are different roles and are treated as context evidence rather than direct detector evidence.

Hybrid content-plus-URL approaches combine message text with URL lexical features, redirects, or destination-page evidence. Systems in the Smishing Detector and DSmishSMS line use this design [2], [54], [79]. The additional signal can help with link-bearing lures, but it adds network and latency considerations and does not cover callback or reply-bait attacks.

Deployment is a separate axis. *Cloud/server-side* inference can support larger models but exports message content and depends on a network. *On-device* systems keep the classification path local, as in SMSAssassin and a character-level evasion-resistant prototype [56], [80]. *Mobile client–server* systems use a handset application while sending features or messages to a remote service; SMSPROTECT belongs in this category rather than in fully local inference [81]. *Federated* systems distribute training without centralizing raw messages, but privacy still depends on the update threat model. Federated Bi-LSTM, DistilBERT, and PhoBERT studies evaluate non-independent and non-identically distributed (non-IID) client data [31], [82], [83]. Most decoder-model SMS studies in the eligibility-filtered subset do not report on-device measurements; compression is a candidate approach, not yet a demonstrated solution for the target task.

Three additional fields are used in the comparison: *language and script*, *explanation*, and *robustness*. Explanation includes instance-level attribution or a generated rationale attached to the verdict, not merely the fact that a model family is inspectable. Robustness records an explicit evasion, adversarial, or drift evaluation [55], [56], [84]. "Not reported" is kept separate from a demonstrated absence of capability.

V. CLASSICAL MACHINE LEARNING AND FEATURE ENGINEERING

A. THE BENCHMARK ERA

Content-based SMS filtering predates smartphones. An early study applied bag-of-words classifiers to operator-collected messages and showed that text categorization could be adapted to short messages [18]. The later SMS Spam Collection contains 5,574 messages—4,827 ham and 747 spam—assembled from the Grumbletext complaint forum, the NUS SMS Corpus, and earlier academic collections, with standard classifier baselines [15], [85]. Follow-up analyses identified substantial near-duplication in the spam class, creating a risk of inflated performance when related messages cross a random split [86], [87]. The collection remains widely reused, so its age and duplicate structure must be considered when interpreting results.

B. RULES AND EARLY SMISHING SYSTEMS

Rule-based systems encode patterns over message text, URLs, and behavioral features. Examples include a nine-rule framework [88], Smishing-Classifier [16], the S-Detector pipeline with a naïve Bayes content component [17], and SMIDCA's correlation-ranked features [74]. These approaches are inspectable and inexpensive to execute, but require manual maintenance and have limited evidence under paraphrase and campaign drift.

SMSPROTECT combines an Android client with an application programming interface to a rule-based machine-learning service [81]. It is a useful operational artifact, but its published description does not establish fully local handset inference or report handset latency, memory, and energy. This distinction is important throughout the review: a mobile application is not necessarily an on-device model.

C. CONTENT AND URL EVIDENCE

Mishra and Soni developed a sequence of hybrid systems following an early survey and content-based detector [89], [90]. Smishing Detector combines message content with URL resolution, redirection, and destination evidence and reports 96.2% accuracy in its evaluation [2]. DSmishSMS refines the same design [54]; Jain et al. report another content-and-URL detector [79]. These systems use information that a text-only

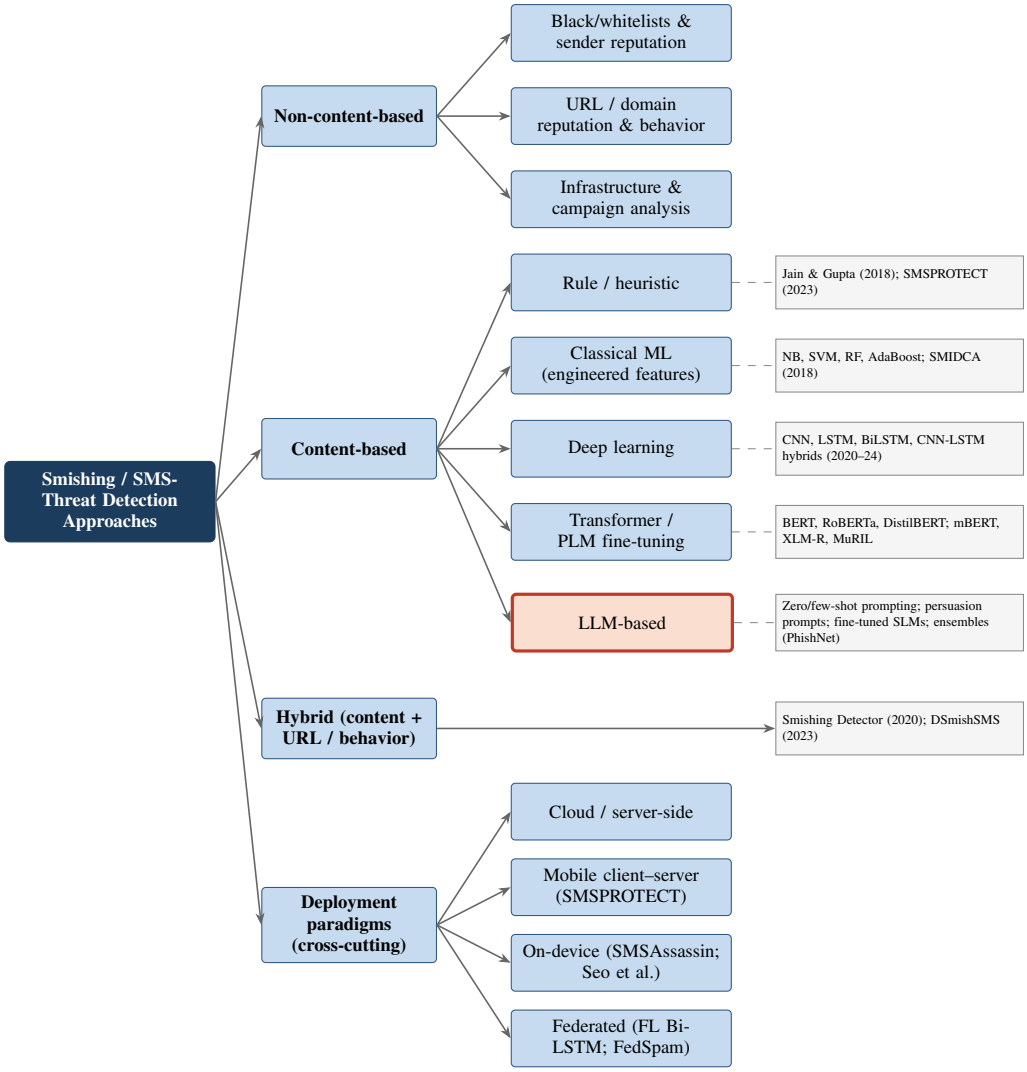

FIGURE 3. Taxonomy of smishing detection approaches.

detector ignores, but they cannot rely on URL evidence for callback, reply-bait, or other link-free lures [5].

D. COMPARATIVE AND OPERATIONAL VARIANTS

Many studies compare classical learners on the canonical corpus or a derivative. They include algorithm suites [91], ensemble and boosting methods [92]–[94], and a back-propagation neural baseline [95]. Text normalization and semantic indexing address abbreviations, slang, and short forms [96], [97]. MDLText examines efficient incremental classification [98]. Because preprocessing, splits, and tuning differ, these papers do not establish a stable cross-paper ranking of classifiers.

Other work changes the operating setting: fog-based processing [99], Bahasa Indonesia filtering [100], joint phishing and smishing checks [101], and an immune-system-inspired mobile-communication framework [102]. TF–IDF with random forests remains a useful baseline in more recent comparisons [103], [104]; its continued competitiveness should be tested on the same data as any more complex model.

Salman et al. contribute a 67,018-message corpus spanning 2012–2023 and test explicit evasive transformations, showing that those transformations degrade the evaluated filters [55]. A 2026 study separately compares feature representations under imbalanced short-text conditions [19]. These results support two limited conclusions. Classical models remain inexpensive and data-efficient, including for Dravidian-language spam [105]. Their performance under new campaigns and adversarial rewriting depends strongly on the representation and evaluation protocol [55]. They should therefore remain mandatory controls for, rather than historical foils to, language-model experiments.

Table 2 summarizes the representative classical and rule-based studies used in the comparison.

VI. DEEP LEARNING APPROACHES

Deep learning replaces most manual feature engineering with learned word- or character-level representations. Convolu-

TABLE 2. Summary of Classical Machine-Learning and Rule-Based SMS/Smishing Detection Studies

Study	Year	Features / approach	Classifier(s)	Dataset	Reported result / key limitation
Gomez Hidalgo et al. [18]	2006	Bag-of-words content features	NB, SVM, others	Operator/UK collections	Feasibility established; pre-benchmark, small corpora
Almeida et al. [15]	2011	Tokenized text features	Suite (SVM strongest)	SMS Spam Collection (5,574; 747 spam)	Benchmark baselines set; near-duplicates, English-only
Joo et al. [17]	2017	Content-monitoring pipeline (S-Detector)	Naïve Bayes	Custom	Device-path blocking design; NB recall limits
Jain & Gupta [88]	2018	Nine message/URL rules	Rule-based	Custom	Transparent filtering; paraphrase-evadable
Goel & Jain [16]	2018	Lexical + behavioral rules	Rule framework	Custom	Mobile-environment focus; rule maintenance burden
Sonowal & Kuppusamy [74]	2018	Correlation-ranked features (SMIDCA)	ML suite	Benchmark SMS	Feature-selection gains; engineered-feature brittleness
Alzahrani & Rawat [91]	2019	Standard text features	Comparative ML suite	SMS Spam Collection	Cross-algorithm comparison; benchmark monoculture
Mishra & Soni [2]	2020	Content + URL behavior (4 stages)	Hybrid ML	Custom + benchmark	96.2% accuracy; blind to link-free lures
Jain et al. [95]	2020	Lexical features	Back-propagation NN	Benchmark	Neural-adjacent baseline; shallow architecture
Bosaeed et al. [99]	2020	Fog-deployed pipeline	NB/SVM variants	SMS Spam Collection	Latency/load balancing; infrastructure complexity
Theodorus et al. [100]	2021	Bahasa Indonesia features	ML suite	Custom Indonesian	Non-English feasibility; small corpus
Mambina et al. [28]	2022	TF-IDF + feature selection	Extra Trees + RF hybrid	32,259 Swahili smishing messages	99.86% accuracy; strong low-resource result, single-country corpus
Jain et al. [79]	2022	Content + URL analysis	ML	Custom	Journal-scale hybrid results; link dependence
Karhani et al. [101]	2023	Domain checker + NLP checker	ML	Custom	Joint phishing/smishing; component coupling
Mishra & Soni [54]	2023	Domain check + content (DSmishSMS)	ML	Custom	Refined hybrid staging; blind to link-free lures
SmishShield [103]	2024	TF-IDF	RF (top performer)	Benchmark	Imbalance-aware evaluation; representation ceiling
Salman et al. [55]	2024	Evasion stress-testing	ML suite	New 67K-message corpus (67,018)	Evasion degrades filters; shallow-model fragility shown
Feature-representation study [19]	2026	Representation ablation	Classical suite	Benchmark (imbalanced)	Representation choice comparable to classifier choice; English-only

tional models capture local patterns; recurrent models represent sequence order; and hybrid systems combine both. For SMS, the potential benefit is tolerance of spelling and surface variation. The costs are larger data requirements and weaker inspectability than rules or linear models.

A. CONVOLUTIONAL AND RECURRENT DETECTORS

Roy et al. compare CNN and LSTM detectors with classical baselines on SMS spam data [20]. Subsequent work includes an Arabic–English CNN–LSTM [21], another CNN–LSTM SMS filter [106], and a joint SMS/email framework [107]. Mishra and Soni implement their earlier Smishing Detector design with an artificial neural network and report 97.40% accuracy in that study [108]. More recent systems include a CNN–Bi-GRU model for the tri-class SMS Phishing Collection [75] and a CNN–LSTM designed to reduce false positives [109].

Character-level processing is useful when attackers or users vary spelling. A Bangla detector combines BERT representations with a character-level CNN and reports 98.47% accuracy on its corpus [30]. Because the system contains a pretrained encoder, it is assigned to the PLM family in aggregate counts even though it is also discussed here.

B. NON-ENGLISH AND DEVICE-ORIENTED EVIDENCE

Recurrent models have been evaluated for Bengali spam [110] and Bangla smishing [111]. Arabic work now includes a hybrid deep-learning smishing detector with dataset construction and explainability analysis [112]. Each result is useful within its own dataset, but the papers do not share a corpus or protocol, so they do not establish a cross-language architecture ranking.

Federated Bi-LSTM training has also been studied for smishing [82]. Its reported 88.78% accuracy should be interpreted only within that evaluation; it does not by itself quantify a general accuracy cost of federation. Seo et al. implement a character-level CNN on a handset and test text-evasion attacks [56]. The study shows that local, robustness-oriented classification is feasible for that compact architecture, but it does not report a complete latency, memory, energy, and precision profile. COPS supplies the fuller named-device precedent discussed in Section X [42].

The deep-learning literature establishes non-English, character-level, federated, and handset examples. It does not show that deep models are uniformly better than classical or PLM systems, because datasets and evaluations vary. Its most transferable design lesson for the target setting is to retain character-level controls whenever Romanization, misspelling, and evasion are expected.

TABLE 3. Summary of Deep-Learning SMS/Smishing Detection Studies. AUC denotes area under the receiver operating characteristic curve.

Study	Year	Architecture	Input representation	Language(s)	Reported result / key limitation
Roy et al. [20]	2020	CNN; LSTM	Word embeddings	English	Outperforms classical baselines; benchmark ceiling
Ghourabi et al. [21]	2020	CNN-LSTM hybrid	Word embeddings	Arabic + English	Beats classical in both languages; two-language scope
Hossain et al. [106]	2022	CNN-LSTM	Word embeddings	English	Confirms hybrid recipe; opacity
Maqsood et al. [107]	2023	DL framework	Learned	English	Joint SMS + email coverage; cross-channel dilution
Mishra & Soni [108]	2022	ANN (Smishing Detector impl.)	Engineered + learned	English	97.40% accuracy; shallow capacity
Mahmud et al. [75]	2024	CNN-Bi-GRU	Word2Vec	English	Multi-class smishing; static embeddings
Mehmood et al. [109]	2024	CNN-LSTM	Learned	English	False-positive reduction focus; single-corpus evaluation
Tanbhir et al. [30]	2024	BERT + char-level CNN	Contextual + character	Bangla	98.47% accuracy; corpus size
Bangla BiLSTM study [111]	2026	Multi-stream BiLSTM	Learned	Bangla	97.89% acc; F1 0.963; AUC 99.73%; single language
Uddin et al. [110]	2020	RNN	Learned	Bengali	Low-resource feasibility; small data
Al Saidat et al. [112]	2026	Hybrid deep model	Learned + explainability analysis	Arabic	Dataset construction and Arabic smishing evaluation; single-language setting
COPS [42]	2024	Disentangled VAE pipeline	Word + character sequences	English	98.15% accuracy; 3.46-MB model, ∼12-MB processing memory, ∼0.08 ms/character on Galaxy S21 FE
Seo et al. [56]	2024	Character-level CNN (on-device)	Character	Unspecified	Evasion-resistant handset prototype; resource profile incomplete
Federated Bi-LSTM [82]	2024	Bi-LSTM (federated)	Learned	English	88.78% accuracy; no controlled centralized comparison

VII. TRANSFORMER AND PRETRAINED LANGUAGE MODEL APPROACHES

Pretrained bidirectional encoders adapt a representation learned from large unlabeled corpora to a smaller labeled classification task [65]. Cross-lingual pretraining extends that transfer across languages [113]. Relevant baselines include mBERT and XLM-R [66], RoBERTa [114], and DistilBERT, which reports retaining approximately 97% of BERT's performance on the General Language Understanding Evaluation (GLUE) benchmark while using 40% fewer parameters [67]. Those general results do not substitute for SMS-specific device measurements.

A. SMS SPAM AND SMISHING DETECTORS

SM-Detector applies BERT fine-tuning to mobile smishing messages [22]. ExplainableDetector fine-tunes RoBERTa and reports 99.84% accuracy on its evaluated SMS spam data, together with Local Interpretable Model-Agnostic Explanations (LIME) and transformer-attribution analysis [23]. Jain et al. compare contextual and TF–IDF representations for a tri-class problem and study decomposition into two binary decisions, while noting model-size concerns for mobile use [115]. Hassan et al. organize transformer-attention signals into textual, structural, and behavioral concepts [116].

Data augmentation is another use of pretrained models. Islam and Munoz use GPT-2-generated minority-class messages with BERT, ELECTRA, and DistilBERT representations in a multi-class pipeline [51]. The method addresses class imbalance, but synthetic-data provenance and leakage must be reported. Email studies with DistilBERT and LIME [117] or a tri-class transformer [118] are transfer evidence; they are dagger-marked and excluded from direct SMS counts in Table 4.

Non-English PLM evidence includes the Bangla BERT–character-CNN system [30] and federated PhoBERT for Vietnamese smishing [31]. FedSpam distributes DistilBERT training for SMS spam [83]. These studies establish feasibility within their own languages and data. They do not yet provide a common multilingual benchmark.

B. INDIC ENCODERS

MuRIL is pretrained on sixteen Indian languages, English, and transliterated counterparts [62]. IndicBERT is a compact ALBERT-style model trained on IndicCorp [63]. Both are appropriate baselines for the proposed evaluation, but neither is evaluated on a fraud-labeled, code-mixed Indian SMS corpus in the reviewed evidence. Comparisons on other tasks show that language-specific Hindi and Marathi encoders can outperform multilingual models in their home language [119], [120]. This supports testing both broad multilingual and language-specific baselines rather than assuming one will transfer best.

PLMs report high scores on the canonical English corpus, but those scores do not resolve dataset age, duplicate leakage, Romanized text, or adversarial evaluation. Text attacks can severely degrade BERT-class models in general [84], while SMS-specific work also shows evasion risk [55]. The evidence supports PLMs as strong candidates, not as a settled answer for code-mixed on-device detection.

TABLE 4. Summary of Transformer / Pre-Trained Language Model Detection Studies

Study	Year	Backbone	Task framing	Language(s)	Explainability	Reported result / key limitation
Ghourabi et al. [22]	2021	BERT (SM-Detector)	Smishing binary	English	No	Early PLM template; robustness untested
BERT smishing study [115]	2025	BERT	Tri-class to dual binary	English	No	Contextual beats TF-IDF; mobile footprint concern
Uddin et al. [23]	2025	Optimized RoBERTa	Spam binary	English	LIME + attribution	99.84% accuracy; saturated benchmark
Hassan et al. [116]	2026	BERT attention concepts	Smishing	English	Concept-level	Efficient explainable design; early-stage evaluation
Chain-transformer study [51]	2025	GPT-2 aug. + BERT/ELECTRA/ DistilBERT	Multi-class	English	No	Minority-class gains; synthetic-data validity
Tanbhir et al. [30]	2024	BERT + char-CNN	Smishing binary	Bangla	No	98.47% accuracy; corpus size
Uddin & Sarker† [117]	2024	DistilBERT	Phishing email	English	LIME	Explainable email detector; transfer evidence
Jamal et al.† [118]	2023	Fine-tuned transformer (IPSDM)	Phishing/spam/ham	English	No	Tri-class email detector; transfer evidence
FedSpam [83]	2023	DistilBERT (federated)	Spam	English	No	Decentralized fine-tuning; communication overhead
Federated PhoBERT [31]	2025	PhoBERT (federated)	Smishing	Vietnamese	No	Non-IID handling; single language

†Adjacent transfer evidence; excluded from the direct SMS detector counts.

VIII. DECODER LANGUAGE MODELS: DETECTION, AUGMENTATION, AND EXPLANATION

Decoder language models appear in several roles that should not be combined in one denominator. They can classify a message, generate training data, explain a verdict, support an ensemble, or generate attacks. Only studies with message-level SMS detection results enter the eligibility-filtered direct-detector counts.

A. PROMPTED CLASSIFICATION

Email and webpage studies provide useful transfer evidence. Heiding et al. evaluate language models as both phishing-email detectors and generators [24]. Ji and Kim show that prompt, modality, and decoding configuration can change webpage-phishing results [76]; LLMPEA reports prompt-injection susceptibility and uneven multilingual behavior for email classification [121]. Structured email and webpage prompting systems [122], [123] and persuasion-oriented feature extraction for spear-phishing [124] further illustrate possible designs. These dagger-marked rows are not SMS results.

Direct SMS evidence is more recent. SpaLLM-Guard compares zero-shot, few-shot, chain-of-thought, and fine-tuned models for SMS spam and evaluates adversarial manipulation and concept drift [34]. Sanjari et al. present preliminary poster evidence for few-shot classification with explanations [27]. Wang et al. study an LLM-agent interface that walks users through an SMS verdict [26]. Hur et al. add English and Korean SMS experiments across commercial and local 2.4–8B models, including prompting, explanations, adversarial edits, drift, cost, and inference-time analysis [77]. The device setting and measurement protocol must still be checked before treating inference time as handset evidence.

B. DATA GENERATION AND ADAPTATION

Shim et al. condition synthetic messages on persuasion categories and evaluate the resulting augmentation for smishing detection [25]. This design makes the intended social-engineering cue explicit, unlike unconstrained generation, but synthetic examples still require provenance, human validity checks, and an authentic-only test set. The earlier GPT-2 augmentation pipeline in [51] addresses the same scarcity problem with a different generator. AbuseGPT demonstrates the dual-use risk by generating smishing campaigns [8]; it is threat context, not a detector.

Task adaptation now includes compact models. KorSmishing Explainer adapts a Korean-centric model with fewer than five billion parameters and evaluates classification and rationales, reporting an improvement over GPT-4 within that study's protocol [29]. Krawczyk et al. fine-tune XLM-R with LoRA for English, Bengali, and Swahili SMS [43]; because XLM-R is an encoder, this study is coded with PLMs rather than decoder LLMs. Agentic Knowledge Distillation trains Qwen2.5-0.5B and SmolLM2-135M students with LoRA for SMS threat detection [32]. It directly weakens any broad claim that fine-tuned small language model (SLM) detectors do not exist, while leaving low-bit Indic and measured handset deployment open.

C. ENSEMBLES, AGENTS, AND CONTEXT EVIDENCE

PhishNet combines a fine-tuned RoBERTa with open-weight language models and synthetic data for ham/spam/smishing classification [78]. The reported gain belongs to that dataset and serving design; it does not establish a general ensemble advantage. Adjacent work includes multimodal language models for phishing webpages [125] and SmishViz for campaign analysis rather than individual-message classification [126]. Table 5 marks campaign monitoring, attacker generation, and non-SMS channels as context or transfer evidence.

Decoder models offer prompting, natural-language rationales, and low-data adaptation. Their evaluation also introduces configuration sensitivity, training-data contamination concerns, and prompt injection for instruction-following components [76], [121]. The reviewed evidence includes direct English, Korean, Bengali, and Swahili SMS language-model results, but no decoder model is jointly evaluated on code-mixed Indian SMS, at a stated low-bit deployment precision, under adversarial tests, with latency, memory, and energy on a physical handset.

IX. CODE-MIXED AND LOW-RESOURCE INDIAN-LANGUAGE PROCESSING FOR SECURITY TASKS

This section addresses the review's central question: what does the research record contain at the intersection of Indian-language text processing and SMS-borne threats? It first maps the corpus and modeling landscape for code-mixed Indic text, then examines the security-specific literature in Indian languages, and finally assesses the limited contact between these otherwise mature strands.

A. CORPORA AND THE MEASURED DIFFICULTY OF THE REGISTER

The empirical foundation for code-mixed Indic processing was laid by shared tasks and purpose-built corpora. SemEval-2020 SentiMix established Hindi–English sentiment as a benchmark and documented ambiguous language identity, unbounded spelling variation, and punctuation noise [59], [127]. DravidianCodeMix then supplied more than 60,000 Tamil–English, Malayalam–English, and Kannada–English comments, including intra-word switching [12]. HASOC-DravidianCodeMix brought mBERT, DistilBERT, and MuRIL to Tamil and Malayalam offensive-language identification [128].

Malayalam has dedicated infrastructure at both word and sentence granularity: a six-label word-level language-identification corpus with transformer baselines [13], sentence-level Malayalam–English classification reaching 99.76% [129], and earlier code-mixed comment classification [130]. The register is therefore characterized and resourced, but for non-security tasks. The gap is application transfer, not the absence of language technology.

B. FOUR STRATEGIES FOR HANDLING CODE-MIXED TEXT

The modeling literature organizes into the four strategies of Fig. 4. *Normalization* transliterates Romanized input back to native script before classification, resting on the Dakshina lexicons [60] and, at scale, on Aksharantar, 26 million attested transliteration pairs across 21 languages, in which Malayalam contributes the single largest training split at 4.1 million pairs, and its IndicXlit model [61]. Recent benchmarking shows general-purpose LLMs approaching but not consistently beating such specialized transliteration models [131], and combining native-script, Roman-script, and translated views yields measurable gains, with the HopeCap capsule network reporting $+6.58\%$ weighted-F1 for Malayalam hope-speech detection by averaging over the three views [132]. The strategy's premise is corroborated from the failure side: controlled evaluations show multilingual PLMs suffering systematic deficits on transliterated Hindi and Malayalam relative to native script, with representation analyses locating the damage in fragmented subword geometry [14], evidence that naive PLM transfer onto Romanized SMS will underperform, and that any Indic-smishing benchmark must treat script as an experimental variable.

Direct multilingual encoding is the second strategy. MuRIL is pretrained on sixteen Indian languages, English, and transliterated counterparts, making it particularly relevant among the surveyed baselines for Romanized input [62]. IndicBERT offers a compact ALBERT-style alternative [63], while XLM-R supplies a broad multilingual reference [66]. IndicXTREME compares models across nine tasks and twenty languages [133]. Language-specific Hindi and Marathi encoders outperform the tested multilingual models on some home-language tasks [119], [120]; this result supports controlled comparison rather than a general claim that monolingual or multilingual pretraining is superior. MuRIL ensembles [134] and sentence-encoder variants [135] provide further adjacent baselines.

Code-mix-specialized pre-training is the third strategy. HingBERT-family models use Hindi–English code-mixed text [136], and Mixed-Distil-BERT extends this approach to Bangla–English–Hindi data [137]. Other approaches add word-level language tags [138] or generate controlled code-mixed examples [139]. The reviewed sources include no Malayalam–English code-mix-specialized encoder, leaving a concrete resource gap.

LLM prompting is the fourth and least settled strategy. A 28-model audit finds uneven Indic coverage [140]; IndicMMLU-Pro reports lower performance than English across nine Indian languages for the models it evaluates [141]; and tested prompting configurations trail fine-tuned encoders on some Tamil code-mixed sentiment and offensive-language tasks [142]. These adjacent results justify task-specific testing; they do not establish a general ranking for smishing.

C. SECURITY INSTANTIATIONS IN INDIAN LANGUAGES

Against this rich processing landscape, the Indian-language SMS-security record is thin and largely classical. Early work extended the English benchmark with Indian-user messages [143], followed by Hindi and Bengali spam typed in Roman script [144]. These studies establish that the target register exists in SMS data, but retain binary spam labels and pre-PLM modeling.

The sustained Ramanujam–Abirami program contributes a 7,700-message Tamil, Telugu, Kannada, and Malayalam corpus [58]; hand-crafted classification [105]; explainable feature selection [145]; graph-based and non-contextual feature evaluation [146], [147]; SMSDHL with Hindi and English [148]; and a 3,894-message Hindi corpus [149]. Collectively, this program supplies multilingual data and efficient base-

TABLE 5. Summary of LLM-Based Threat-Detection Studies

Study	Year	Model(s)	LLM role	Channel	Language(s)	Key finding / key limitation
Heiding et al.† [24]	2024	GPT, Claude, PaLM, LLaMA	Zero-shot classifier + generator	Email	English	Dual-use email evidence; transfer and threat context
Ji & Kim† [76]	2025	7 LLMs vs 2 DL baselines	Prompted classifier	Websites	English	Configuration swings rankings; transfer evidence
LLMPEA† [121]	2025	GPT-4o, Claude Sonnet 4, Grok-3	Prompted classifier	Email	Multilingual probes	Prompt-injection and multilingual transfer evidence
ChatSpam-Detector† [122]	2024	LLM (structured prompting)	Prompted classifier	Email	English	Structured prompting; transfer evidence
ChatPhish-Detector† [123]	2024	LLMs	Prompted classifier	Websites	English	Website detector; transfer evidence
Prompted contextual vectors† [124]	2024	LLM ensemble	Feature extractor	Email (spear)	English	Persuasion-question features; transfer evidence
Shim et al. [25]	2024	Few-shot prompted LLM	Persuasion-guided augmenter	SMS	English (Korean context)	Theory-grounded augmentation improves scarce-data detection; augmentation-side only
KorSmishing Explainer [29]	2024	Korean LLM (<5B) + GPT-4	PEFT classifier + explainer	SMS	Korean	15% accuracy gain over GPT-4; explanation quality evaluated; Korean-only
SpaLLM-Guard [34]	2025	GPT-4, DeepSeek, LLaMA-2, Mixtral	Prompted + fine-tuned classifier	SMS	English	Mixtral reaches 98.61%; evaluates adversarial manipulation and concept drift; no edge profiling
Sanjari et al. [27]	2025	Small language models	Few-shot classifier + explainer	SMS	English	Few-shot smishing detection with explanations; poster-scale evaluation
SOUPS LLM agents [26]	2025	LLM agents	Agentic explainer	SMS	English	Stepwise user-facing rationale for verdicts; latency/cost of agent loops
PhishNet [78]	2026	RoBERTa + 3 open LLMs	Ensemble (dual voting)	SMS	English	∼96% to 98.5% via ensemble + T5 augmentation; serving cost of ensemble
Hur et al. [77]	2026	Commercial + local 2.4–8B LMs	Prompted classifier + explainer	SMS	English / Korean	Drift, adversarial edits, explanations, cost, and inference-time evidence; no full handset energy profile
Agentic knowledge distillation [32]	2026	Qwen2.5-0.5B; SmolLM2-135M	Distilled + LoRA-adapted SLM detector	SMS	English	Compact SMS students; no code-mixed Indic or measured handset-energy result
MLLM webpage study† [125]	2024	Multimodal LLMs	Brand-ID + verification	Websites	English	Different channel and modality; transfer evidence
LoRA multilingual smishing [43]	2026	XLM-R + LoRA	PEFT fine-tuning	SMS	En/Bengali/ Swahili	Up to 97% F1 from 50–250 samples/language; encoder-only; no quantization; no Indic; no on-device profiling
AbuseGPT† [8]	2024	Commercial chatbots	Attacker-side generator	SMS	English	Threat context; not a detector
SmishViz† [126]	2025	Graph analytics (+LLM-era data)	Campaign monitoring	SMS	English	Campaign context; not message-level detection

†Transfer or context evidence; excluded from the direct SMS detector counts. XLM-R+LoRA [43] is counted with PLMs in the generation analysis.

lines but does not isolate smishing or test contextual Indic encoders.

Adjacent security tasks include MPNet–CNN fusion for Dravidian offensive language [150] and transformer baselines on DravidianCodeMix [12]. A Filipino–English smishing study reports a 99.20% F-score with TF–IDF and a support vector machine [151]. Because its language pair, data, and protocol differ, the result does not establish expected performance for Indian code-mixed SMS; it shows only that code-switched smishing evaluation is possible in another setting.

D. LIMITED INTEGRATION BETWEEN INDIC NLP AND SMISHING DETECTION

Code-mixed Indic NLP provides large transliteration resources [61], purpose-built encoders [62], [63], [136], and language-model audits [140]. Indian-language SMS security provides corpora and several direct detectors, mainly with hand-crafted features and classical learners [146]. The two areas overlap in Roman-script SMS spam, but not yet in the full fraud-specific task. Within the stated sources and date window, the search did not identify a Malayalam or code-mixed Indian smishing corpus, MuRIL or IndicBERT fine-tuning on fraud-labeled Indian SMS, or a decoder-model evaluation on this register.

Malayalam–English is one defensible first case study because Malayalam has Aksharantar's largest reported transliteration split [61], word- and sentence-level language-identification corpora [13], [129], and a sizable code-mixed classification precedent [12]. This choice would not justify claims about all Indian languages; later work should test

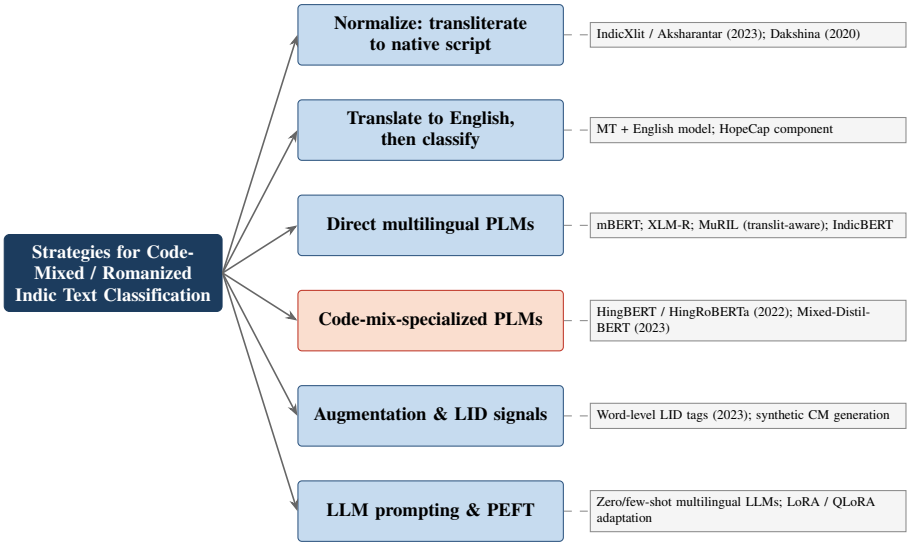

FIGURE 4. **Strategies for classifying code-mixed and Romanized Indic text.**

additional language families and scripts.

Table 6 consolidates the Indian-language and code-mixed studies most relevant to SMS threat detection.

X. RESOURCE EFFICIENCY AND ON-DEVICE DEPLOYMENT

Efficiency is part of the SMS threat model. Exporting a private message to a cloud classifier creates a data-flow and network dependency, while local inference must fit a battery-powered handset. Ordinary message arrival is a sparse, batch-one workload; burst processing may additionally expose thermal limits. Both conditions should be measured rather than inferred from generative mobile benchmarks. Fig. 5 organizes the relevant methods.

A. POST-TRAINING QUANTIZATION AND ADAPTATION

Post-training quantization (PTQ) compresses weights, and sometimes activations, without full retraining. GPTQ performs layer-wise weight quantization [35]; activation-aware weight quantization (AWQ) protects salient channels [36]; SpQR treats outliers separately [152]; QuantEase uses coordinate descent [153]; and QuIP explores two-bit weights [154]. Activation-oriented methods include LLM.int8() [71], SmoothQuant [70], ZeroQuant [155], and OmniQuant [156]. Their suitability depends on the model, calibration data, bit format, group size, runtime kernels, and target hardware.

One general-purpose study across models from 2B to 405B parameters reports that four-bit weight-only PTQ often preserves much of the evaluated task performance [41]. This supports four-bit inference as a useful experimental baseline, not a task-independent default for security text.

Low-rank adaptation (LoRA) trains small adapter matrices over a frozen model [72]. Quantized LoRA (QLoRA) backpropagates through a frozen four-bit NormalFloat (NF4) backbone using double quantization and paged optimizers [37]. QLoRA reduces training memory; it does not by itself state the precision or kernel used after adapters are merged or served. Smishing-specific parameter-efficient fine-tuning (PEFT) has been evaluated on an unquantized XLM-R encoder [43], while Agentic Knowledge Distillation adds LoRA-adapted compact decoder students [32]. A controlled comparison of full-precision adaptation, quantized training, and low-bit mobile inference for code-mixed Indic smishing remains unavailable.

B. DISTILLATION AND COMPACT MODELS

Knowledge distillation transfers behavior from a teacher to a smaller student [69]. DistilBERT reports retaining about 97% of BERT's General Language Understanding Evaluation (GLUE) benchmark performance with 40% fewer parameters [67]. Later compact examples include Gemma 2 [39], the phone-oriented Phi-3-mini technical report [38], Qwen2.5 [40], MobileLLM [157], TinyLlama [158], and Gemma 3's local–global attention for reducing key–value-cache growth [159]. SLM surveys cover the larger design space [48]. These models are reproducible reference points, not a claim that a fixed product list will remain current.

C. WHAT DEVICE MEASUREMENT REQUIRES

Mobile benchmarks provide useful instruments. MELTing Point measures performance and energy on mobile hardware [160]; MobileAIBench reports time to first token, throughput, resident memory, and battery drain [33]; ELIB studies throughput and memory-bandwidth utilization across edge platforms [161]; and broader work maps edge generative-AI architectures [162]. Sustained generative workloads can exhibit thermal throttling [73], while mobile dynamic voltage and frequency scaling (DVFS) settings can change latency and energy [163]. For a detector, end-to-end batch-one classification latency is primary; time to first token matters only when a rationale is generated. A burst test should be reported

TABLE 6. Indian-Language and Code-Mixed Studies Relevant to SMS Threat Detection

Study	Year	Task	Language(s) / script	Method family	Data scale	Key finding / gap left open
Agarwal et al. [143]	2015	SMS spam	English + Indian-user messages	Classical ML	Extended benchmark	Earliest Indian-user extension of the canonical corpus; binary labels, pre-PLM
TIU Indian spam [144]	2019	SMS spam	Hindi/Bengali in Roman script + En	Classical ML	Extended corpus	First Roman-script Indic spam extension; no fraud/smishing split
Ramanujam & Abirami [58]	2023	SMS spam corpus	Ta/Te/Kn/Ml (+Roman-script Ta)	Expert-labeled dataset	∼7,700 msgs	Reference Dravidian spam corpus; spam/ham only, no smishing class
Ramanujam et al. [105]	2024	SMS spam	Ta/Te/Kn/Ml	Hand-crafted features + ML	7,700 msgs	Best-performing method among those compared; pre-PLM scope
Thirumalai et al. [145]	2024	SMS spam	Dravidian	XAI feature selection	Multi-corpus	Explainable feature choice; classical scope
Meta-data features [146]	2025	SMS spam	Dravidian	Graph-based feature eval	Multi-corpus	Efficient alternative to DL/LLM cost; accuracy ceiling unexamined vs PLMs
NCMDFE [147]	2025	SMS spam	Multiple Indian languages	Non-contextual meta-features + RF	6 datasets	Language-agnostic feature viability; contextual semantics ignored
SMSDHL [148]	2025	SMS spam corpus	Ta/Te/Kn/Ml/Hi/En	Dataset	Multi-language	Broadest Indic spam coverage; no smishing labels
Hindi spam corpus [149]	2024	SMS spam corpus	Hindi	Dataset	3,894 msgs	Real-user Hindi collection; binary labels only
DravidianCode-Mix [12]	2022	Sentiment / offensive	Ta-En, Ml-En, Kn-En code-mix	Corpus + ML/DL baselines	60K+ comments	Reference CM corpus incl. 20K Ml-En; non-security tasks
HASOC-DravidianCode-Mix [128]	2021	Offensive language	Ta, Ml (code-mix)	mBERT/ DistilBERT/ MuRIL entries	Shared task	Multilingual encoders audition on Dravidian CM; no SMS/threat framing
Thara & Poornachandran [13]	2021	Word-level LID	Ml-En code-mix	Transformers	6-label corpus	High in-domain scores reported; upstream task only
Sentence-level Ml-En LID [129]	2024	Sentence LID	Ml-En	BERT/ DistilBERT	Custom	99.76% accuracy on one dataset; upstream task only
Aksharantar / IndicXlit [61]	2023	Transliteration	21 languages; Ml largest split	Seq2seq + corpus	26M pairs (4.1M Ml)	Normalization infra at scale; not applied to security text
Transliterated Hi/Ml study [14]	2022	Cross-lingual classification	Romanized Hi/Ml	mBERT/XLM-R probing	Benchmark	Systematic PLM deficit on Roman script; predicts naive-transfer failure
MPNet+CNN offensive [150]	2023	Offensive language	Dravidian CM	PLM fusion	DravidianCodeMix	Modern toolbox works on Dravidian CM; not applied to smishing

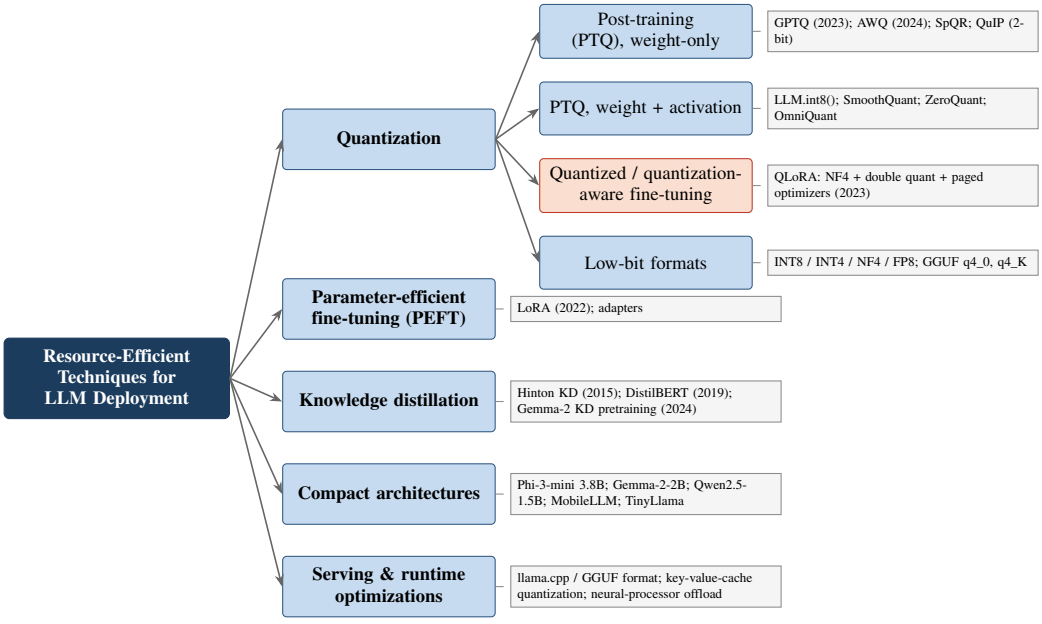

FIGURE 5. Taxonomy of resource-efficiency techniques for language-model deployment.

separately from ordinary sparse arrival.

D. SMS SYSTEMS AND SECURITY BOUNDARIES

SMSAssassin is an early handset filtering system [80]; SMSPROTECT is a mobile client connected to a remote rule-based service rather than evidence of embedded local inference [81]; and Seo et al. implement a character-level CNN on a handset and test text evasion [56]. COPS provides the most detailed named-device detector row in the coded subset: on a Samsung Galaxy S21 FE it reports a 3.46-MB model, about 12 MB of processing memory, and about 0.08 ms per character [42]. Federated DistilBERT [83], Bi-LSTM [82], and PhoBERT [31] keep raw training messages decentralized, but do not automatically protect the information carried by model updates.

Model extraction studies also show that models embedded in Android applications can be recovered and attacked [164], [165]. A deployment claim should therefore specify where preprocessing and inference occur, what data or updates leave the phone, and which resource and security properties were measured.

Deployment measurement is an additional unresolved bottleneck. Candidate components have been evaluated separately: four-bit PTQ on general benchmarks [41], QLoRA for memory-efficient adaptation [37], compact model families [38]–[40], mobile profiling protocols [33], [73], and COPS on a named handset [42]. No reviewed detector combines adversarial code-mixed Indic SMS evaluation with stated precision, latency, memory, and energy. Direct tests must determine whether compression changes adversarial fragility [55], [84] or disproportionately affects rare and fragmented subwords.

Table 8 compares the methods and their unresolved deployment questions.

XI. DATASETS, BENCHMARKS, AND EVALUATION PROTOCOLS

A. THE ENGLISH CANON AND ITS LIABILITIES

The SMS Spam Collection contains 5,574 messages, including 747 spam, and draws from the Grumbletext complaint forum, the NUS corpus of legitimate student messages, and earlier academic collections [15], [85], [166]. Its creators document near-duplication in the spam class, which creates leakage risk when related examples cross random splits [86], [87]. Its 2011 sampling also predates many current delivery-scam and generative-text patterns. Salman et al. provide the largest public labeled SMS-threat corpus identified in this review, with 67,018 messages and an evasion-aware evaluation [55]. Mishra and Soni provide a 5,971-message tri-class dataset containing 4,844 ham, 489 spam, and 638 smishing messages [49], [50]. SmishTank crowdsources live reports [167], and later work uses clustering and dataset fusion to construct a multi-class corpus [168]. Version, source overlap, and independent content should be checked before treating a fused dataset's row count as new evidence.

B. BEYOND ENGLISH: THE EMERGING INDIC AND SOUTH-ASIAN SHELF

The non-English shelf is young but real. BangalaBarta distinguishes smishing, promotional, and normal messages in Bangla collected in a Bangladeshi telecom context [169], complementing the corpora underlying recent Bangla detectors [30], [110], [111]. For Indian deployments, the Ramanujam–Abirami program supplies the core: the 7,700-message Dravidian collection spanning Tamil, Telugu, Kannada, and Malayalam, including Tamil written verbatim in Roman script [58]; SMSDHL, extending coverage to Hindi and English [148]; and a 3,894-message Hindi collection gathered from real users [149]. Adjacent resources complete the experimental toolkit even though they carry no threat labels: DravidianCodeMix for code-mixed classification precedent [12], word- and sentence-level Malayalam–English language-identification corpora [13], [129], and the Dakshina and Aksharantar transliteration collections for normalization [60], [61]. The reviewed Indian SMS datasets remain spam/ham labeled, and BangalaBarta is single-language and native-script. No public corpus was identified that separates ham, bulk spam, and smishing in code-mixed Indian SMS.

Table 9 catalogues the public datasets discussed in this section.

Table 9 combines exact counts with documented approximate values, including approximately 7,700 messages for the Dravidian corpus. A precise median or mean across the table would therefore imply unsupported numerical comparability. Size and task adequacy also differ. The reviewed Indian datasets are binary spam/ham collections; tri-class labels are available in English and in single-script Bangla from Bangladesh, but not in the target code-mixed Indian setting. Dataset version, independent message count after deduplication, label ontology, provenance, and language/script coverage are more informative than a raw total alone. Section XII tests that claim directly.

C. EVALUATION PROTOCOLS: FIVE SYSTEMIC WEAKNESSES

Reading evaluation practice across the comparative tables reveals five recurring weaknesses. *(1) Imbalance-blind metrics.* Accuracy on corpora that are 85–95% ham rewards the majority class; macro-averaged F1 and per-class recall on the smishing class are reported inconsistently, so headline figures are not commensurable. *(2) Duplicate-contaminated splits.* Given the documented near-duplication in the canonical corpus [86], random splits leak; few papers deduplicate before splitting. *(3) In-distribution myopia.* Cross-dataset evaluation is rare, so reported performance conflates task learning with corpus memorization.

(4) Limited robustness reporting. The eligibility-filtered table subset contains four direct SMS studies with explicit evasion, adversarial, or drift evaluation [34], [55], [56], [77]. Instruction-following detectors additionally require prompt-injection tests [121] and complete reporting of prompt, model

TABLE 7. Hardware-Grounded Profiling Evidence Available to an On-Device Smishing Benchmark

Study / benchmark	Workload and hardware reporting	Metrics demonstrated	Missing for the target detector	Methodological value
COPS [42]	Smishing/URL-phishing on Samsung Galaxy S21 FE	Model size, processing memory, inference time per character	Energy, numerical precision, sustained-load behavior	Direct detector precedent; only named-handset time-and-memory row
MELTing point [160]	Mobile LLM inference across device/model settings	Performance and energy profiling	SMS classification and adversarial workload	Establishes that energy must be measured rather than inferred from model size
MobileAIBench [33]	Instrumented application on real handsets	Time to first token, per-token throughput, resident memory, battery drain	Per-message classification protocol and smishing accuracy	Reusable measurement vocabulary and app-level instrumentation
ELIB [161]	Language-model inference across edge platforms	Throughput and model-bandwidth utilization	Security task, per-class utility, attack conditions	Separates compute and memory-bandwidth bottlenecks
Sustained-load study [73]	4-bit 1.5B model on a flagship phone	Warm-up versus steady-state throughput, thermal throttling	Detector-specific latency and battery cost	Shows why first-minute latency is not a deployment result
Mobile DVFS study [163]	Mobile LLM inference under CPU/GPU governor settings	Latency and energy sensitivity to power policy	SMS workload and fixed-governor reporting convention	Identifies a hardware-control variable that otherwise confounds comparisons

TABLE 8. Resource-Efficiency Techniques for Language-Model Deployment

Technique	Type	Typical precision	Retraining	Key mechanism	Evidence base	Fit for on-device SMS filtering
GPTQ [35]	PTQ, weight-only	3–4 bit	No	Layer-wise 2nd-order quantization	Generative LLMs	High (footprint), untested on security text
AWQ [36]	PTQ, weight-only	4 bit	No	Protects activation-salient channels	General LLM benchmarks [41]	Candidate; kernel and hardware dependent
SpQR [152]	PTQ, weight-only	∼3–4 bit + sparse	No	Outliers kept high-precision	Near-lossless compression	Moderate (kernel support)
QuIP [154]	PTQ, weight-only	2 bit	No	Incoherence processing w/ guarantees	Lower frontier	Experimental
QuantEase [153]	PTQ, weight-only	3–4 bit	No	Coordinate-descent optimization	GPTQ/AWQ-comparable	Moderate
LLM.int8() [71]	PTQ, W+A	8 bit	No	Outlier dims in FP16	Transformers at scale	Moderate (8-bit floor)
SmoothQuant [70]	PTQ, W+A	8 bit	No	Migrates activation outliers to weights	W8A8 serving	Moderate
ZeroQuant [155]	PTQ, W+A	8 bit	Light (LKD)	Fused kernels + layer distillation	Large transformers	Moderate
OmniQuant [156]	PTQ, W+A	4–8 bit	Calibration	Learned outlier suppression	Broad settings	Moderate
LoRA [72]	PEFT	FP16 backbone	Adapter-only	Low-rank update matrices	Ubiquitous	High (with quantized base)
QLoRA [37]	Quantized fine-tuning	4-bit NF4 backbone	Adapter-only	Double quantization + paged optimizers	General adaptation benchmarks	Candidate for memory-limited training; serving precision separate
Distillation [67], [69]	Model compression	Method-dependent	Full student training	Teacher–student transfer	General encoder/decoder evaluations	Candidate; task-specific validation required
Compact SLMs [38]–[40]	Architecture	Native and quantized variants	N/A	Smaller parameter budgets	General language benchmarks	Candidate backbones, not smishing evidence

TABLE 9. Public Datasets for SMS Threat Detection and Adjacent Code-Mixed Tasks

Dataset	Year	Language(s) / script	Size	Labels	Key property	Limitation
SMS Spam Collection [15], [85]	2011	English	5,574 (747 spam)	ham/spam	Field's canonical benchmark	Age; near-duplicates [86]
NUS SMS Corpus [166]	2013	English + Mandarin (Singapore)	67,093 (2015 release; ∼10K at the canon's 2011 sampling)	legitimate only (no threat labels)	Ham source for the canon	No threat labels
Salman et al. corpus [55]	2024	English	67,018	ham/spam	Largest public; evasion-aware	Binary labels
SMS Phishing Dataset [49], [50]	2022	English	5,971 (638 smishing)	ham/spam/ smishing	Tri-class publication and archived ver. 1 data	English only
Swahili smishing corpus [28]	2022	Swahili	32,259	legitimate/ smishing	Real mobile-money threat messages	Single country and language
SmishTank dataset [167]	2024	English	Crowdsourced	smishing	Live, topic-analyzed lures	Coverage bias
Multi-class fusion corpus [168]	2026	English	Fused multi-source	multi-class	Clustering-based modernization	Fusion heterogeneity
BangalaBarta [169]	2026	Bangla (Bangladesh)	2,772	normal/ promotional/ smishing	Tri-class South-Asian SMS	Single language; heavy exact duplication
Dravidian Spam SMS [58]	2023	Ta/Te/Kn/Ml + Roman-script Ta	∼7,700	ham/spam	First Dravidian SMS corpus; Roman script acknowledged	No smishing class
SMSDHL [148]	2025	Ta/Te/Kn/Ml/Hi/En	Multi-language	ham/spam	Broadest Indic coverage	No smishing class
Hindi Spam SMS [149]	2024	Hindi	3,894	ham/spam	Real-user collection	No smishing class
DravidianCodeMix [12]	2022	Ta-En/Ml-En/Kn-En code-mix	60K+ comments	sentiment/offensive	20K Ml-En code-mix precedent	Not SMS; no threat labels
Ml-En LID corpora [13], [129]	2021–24	Malayalam-English	Word + sentence level	language ID	Upstream tooling for pipelines	Upstream only
Dakshina / Aksharantar [60], [61]	2020–23	12–21 Indic languages	26M pairs (Ml: 4.1M)	transliteration	Normalization at scale	Not security data

version, decoding settings, run count, and contamination controls [76]. *(5) Incomplete deployment reporting.* COPS reports named-handset time and memory [42], but the coded subset contains no detector reporting accuracy, numerical precision, end-to-end latency, resident and peak memory, and energy per message together [33], [73].

Future work should report macro-F1 and per-class precision/recall/F1, smishing precision–recall area under the curve, calibration, false positives per 1,000 legitimate messages, confidence intervals, and multiple random seeds. Splits should be deduplicated and grouped by campaign, sender, template, URL domain, and time, with at least one provenance-audited source- or corpus-held-out test. Dataset documentation should include provenance, version, license, lawful collection basis, consent or waiver, personally identifiable information removal, annotation and adjudication rules, representativeness, and synthetic-data provenance. Any deployment claim should add end-to-end latency, peak and resident memory, and energy on named hardware at the stated precision. Sections XII and XIV instantiate and extend these requirements.

XII. PRELIMINARY VALIDATION OF THE PROPOSED PROTOCOL

This section adds a small empirical anchor to the review. It is a protocol-feasibility proof on proxy corpora for duplicate control, calibration, and cross-corpus leakage; it is not an evaluation of code-mixed Malayalam–English smishing, which remains Gap G2. It does not rank the architectures surveyed above. Instead, it instantiates the simple-control and cross-corpus parts of the proposed benchmark on freely available tri-class English and native-script Bangla SMS corpora, and asks whether duplicate handling changes the conclusion. The complete split indices, per-seed predictions, archive hashes, and executable scripts accompany the supplement.

A. DATASETS AND DUPLICATE AUDIT

The English experiment uses version 1 of the 5,971-row SMS Phishing Dataset (4,844 ham, 489 spam, and 638 smishing) [49], [50]. The Bangla experiment uses BangalaBarta version 3, whose 2,772 rows are evenly divided among normal, promotional, and smishing messages [169]; these labels are mapped to ham, spam, and smishing for common metrics. The repository licenses are CC BY 4.0 and CC BY-NC-SA 4.0, respectively. The downloaded archives matched the SHA-256 values returned by the Mendeley file service.

Text is normalized with Unicode NFKC and collapsed

whitespace; casefolding is used only for duplicate keys. A normalized text assigned more than one label is quarantined rather than resolved from the test data. The first occurrence of each remaining exact duplicate is retained. This leaves 5,797 English messages after removing 106 duplicate rows and quarantining 68 rows from 34 conflicting-label text groups. BangalaBarta reduces to 1,204 messages after removing 1,565 exact duplicates and quarantining three rows from one conflicting-label group. This large difference between published rows and independent texts is itself an evaluation result: row count is not an adequate measure of corpus diversity.

Near duplicates are connected when their TF–IDF character-trigram cosine similarity is at least 0.90. Connected components, not individual rows, define the grouped split. The cleaned English corpus contains 113 multirow components (largest size five); the cleaned Bangla corpus contains 31 (largest size three).

B. SPLITS, MODELS, AND METRICS

Both corpora use 70/15/15 train/validation/test partitions and seeds 13, 29, 47, 71, and 97. The naive English condition is label-stratified at the row level. The grouped conditions search group-preserving partitions for the closest class balance while keeping every near-duplicate component intact. No vocabulary or calibration parameter is fit on a test partition.

The controls are a word TF–IDF linear SVM with one- and two-word features and a character TF–IDF linear SVM with two- through five-character features. Both use class weighting; $C \in \{0.1, 1, 10\}$ is selected by validation macro-F1. Validation-only temperature scaling supplies probabilities for 15-bin expected calibration error (ECE). We report macro-F1, smishing precision/recall/F1, false positives per 1,000 ham messages, and two-sided 95% Student-t intervals across the five fixed splits. The intervals quantify split sensitivity; the five scores are not treated as independent samples from a population of corpora.

C. LEAKAGE SENSITIVITY AND CROSS-CORPUS AUDIT

Table 10 shows that naive splitting permits 50–64 near-duplicate components to cross partitions, whereas grouped splitting permits none. The paired naive-minus-grouped macro-F1 difference is 0.031 [0.003, 0.059] for the word SVM and 0.027 [−0.013, 0.067] for the character SVM. The direction is consistent with leakage inflation, but the character-model interval includes zero. The result therefore supports group-aware evaluation without claiming a universal three-point correction.

BangalaBarta remains separable after aggressive exact deduplication: Table 11 reports grouped macro-F1 of 0.967 for the word SVM and 0.977 for the character SVM. These high scores do not establish an Indic-encoder advantage. They show that the cleaned single-script corpus is executable under the protocol and that false positives on normal messages remain informative even when macro-F1 is high.

The proposed UCI transfer test required an additional provenance check. The UCI SMS Spam Collection has 5,159 unique, conflict-free messages after exact deduplication [15], [85]. Of these, 4,601 are exact matches and 4,751 are exact or near matches to the ham/spam portion of the English training source, consistent with the source dataset's documented reuse of the canonical collection. Only 408 UCI messages (100 ham and 308 spam) remain contamination-clean. On that bounded target, the word SVM loses 0.063 macro-F1, while the character SVM loses 0.006 with an interval spanning zero. An unfiltered source-to-UCI score would therefore be a contaminated evaluation, not evidence of generalization.

These results validate protocol executability and quantify leakage and cross-corpus effects on available proxies; they do not estimate performance on code-mixed Indian-language smishing, which requires the G2 corpus. The held-language-out rotation is not applicable because both primary corpora are single-language. The S3 comparison of MuRIL, IndicBERT, and XLM-R remains a preregisterable extension on the same released Bangla splits. Encoder scores are deliberately withheld because no complete five-seed run satisfying the same split, validation-only tuning and calibration, and per-seed logging protocol was completed before the manuscript freeze. Reporting a single opportunistic run would be statistically non-comparable to Tables 10 and 11; the release therefore supplies executable configurations and the exact splits instead. This scope boundary is not evidence that the classical controls outperform pretrained encoders.

XIII. COMPARATIVE ANALYSIS AND DISCUSSION

The literature is easiest to interpret by asking what evidence each study supplies, rather than by treating model families as stages on a single accuracy ladder. Rules and classical models provide inexpensive, inspectable baselines and several handset implementations [81], [98]. Deep models add learned word- and character-level representations [20], [56]. Pretrained language models (PLMs) improve transfer from unlabeled text and support established attribution tools [22], [23]. Decoder language models add prompting, data generation, and natural-language rationales [25], [26]. These capabilities were evaluated on different data and under different constraints. The review therefore does not infer that one family caused a higher or lower score than another.

A. WHAT THE ELIGIBILITY-FILTERED COMPARISON SET SHOWS

The comparison tables include direct SMS detectors together with clearly marked email and webpage transfer studies, attacker-side generation, and campaign monitoring. Applying the evidence-layer rule in Section III leaves 48 direct SMS studies in the table-based quantitative subset. The counts below are useful for auditing those tables, but the subset is purposive and omits some direct studies discussed elsewhere in the article. It cannot support literature-wide prevalence estimates.

TABLE 10. Five-Seed Tri-Class Results on the SMS Phishing Dataset Under Naive and Near-Duplicate-Grouped Splits

Model	Split	Macro-F1	Smishing precision	Smishing recall	Smishing F1	FP / 1,000 ham	ECE	Clusters crossing partitions
Word TF–IDF SVM	Naive	0.916 [0.904, 0.929]	0.926 [0.902, 0.951]	0.914 [0.865, 0.963]	0.919 [0.900, 0.939]	2.20 [0.25, 4.15]	0.013 [0.010, 0.015]	50–64
Word TF–IDF SVM	Grouped	0.885 [0.859, 0.911]	0.896 [0.862, 0.930]	0.887 [0.812, 0.962]	0.890 [0.843, 0.938]	1.93 [0, 5.27]	0.012 [0.008, 0.016]	0
Character 2–5-gram SVM	Naive	0.921 [0.889, 0.953]	0.924 [0.882, 0.966]	0.919 [0.874, 0.964]	0.921 [0.886, 0.956]	0.28 [0, 1.04]	0.013 [0.009, 0.017]	50–64
Character 2–5-gram SVM	Grouped	0.894 [0.868, 0.919]	0.893 [0.853, 0.932]	0.897 [0.824, 0.969]	0.894 [0.849, 0.939]	0.28 [0, 1.04]	0.011 [0.009, 0.013]	0

Values are means [two-sided 95% Student-t intervals] over seeds 13, 29, 47, 71, and 97. The intervals describe repeated-split sensitivity, not independent replications. Exact duplicates were removed before both conditions. Grouped splits keep every character-trigram TF–IDF cosine component at similarity ≥ 0.90 within one partition. ECE uses 15 equal-width confidence bins after validation-only temperature scaling.

TABLE 11. Grouped Bangla Baselines and Contamination-Screened Cross-Corpus Transfer

Model	Evaluation	Source / in-domain macro-F1	Transfer macro-F1	Macro-F1 drop	Smishing F1	FP / 1,000 ham	ECE
Word TF–IDF SVM	BangalaBarta grouped	0.967 [0.944, 0.989]	–	–	0.973	42.20	0.035
Character 2–5-gram SVM	BangalaBarta grouped	0.977 [0.956, 0.998]	–	–	0.979	17.96	0.024
Word TF–IDF SVM	English source → UCI clean	0.953 [0.929, 0.976]	0.889 [0.873, 0.906]	0.063 [0.048, 0.079]	–	0.00	0.052
Character 2–5-gram SVM	English source → UCI clean	0.963 [0.953, 0.972]	0.956 [0.942, 0.970]	0.006 [−0.009, 0.022]	–	4.00	0.019

BangalaBarta values use the same five seeds and grouped protocol as Table 10; confidence intervals for smishing F1, FP rate, and ECE are reported in the released result file. The UCI target contains 408 contamination-clean messages (100 ham, 308 spam) after removing exact and near overlaps with the English source. Transfer results are therefore a bounded stress test, not an estimate for the full UCI collection.

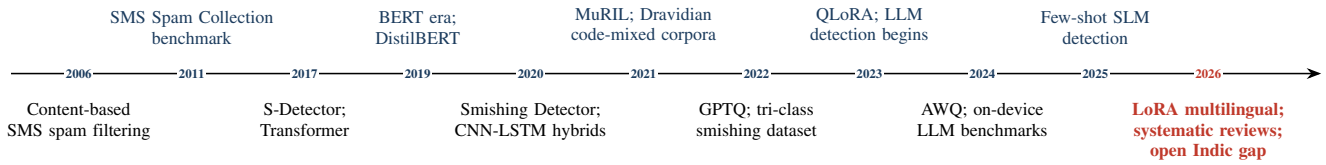

FIGURE 6. Evolution of the field, 2006–2026.

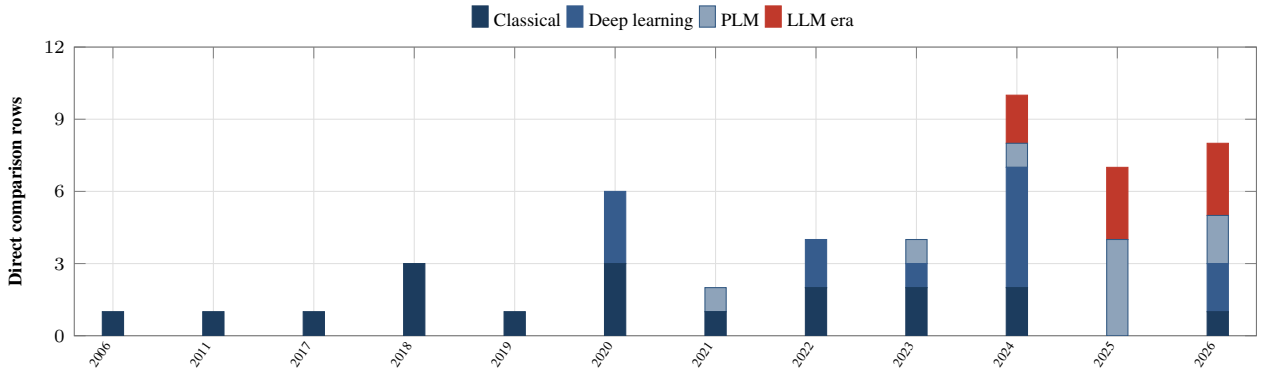

FIGURE 7. Publication-year distribution of the 48 direct SMS studies in the eligibility-filtered table-based comparison set, coded by primary methodological generation. The Bangla BERT–character-CNN hybrid [30] and XLM-R with LoRA [43] are assigned to the PLM generation. Dagger-marked transfer and context rows in Tables 3–5 are excluded. Counts refer to the 48 tabled direct SMS studies; 2026 is partial through 12 July.

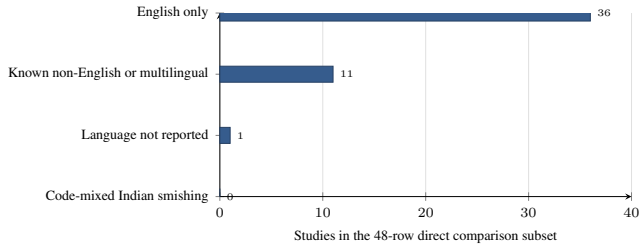

FIGURE 8. Language reporting in the eligibility-filtered 48-study table subset. Thirty-six studies (75.0%) are English-only, eleven include a known non-English or multilingual evaluation, and one does not report language clearly. Counts refer to the 48 tabled direct SMS studies. Indian-language SMS spam studies discussed in Section IX are not all represented in the generation tables.

Language reporting. Thirty-six of the 48 rows (75.0%) evaluate English alone, eleven (22.9%) include a known non-English or multilingual setting, and one (2.1%) does not state the language clearly (Fig. 8). The non-English rows include Bangla or Bengali [30], [110], [111], Korean [29], [77], Swahili [28], Vietnamese [31], Bahasa Indonesia [100], Arabic or Arabic–English [21], [112], and multilingual English/Bengali/Swahili evaluation [43]. Direct Indian-language SMS work exists outside these generation tables [105], [143]–[147], but it uses spam/ham labels. No reviewed code-mixed Indian-language SMS corpus separates smishing from bulk spam.

Explanation and robustness. Seven rows (14.6%) attach an explanation mechanism to a detector verdict or evaluate explanation output [23], [26], [27], [29], [77], [112], [116]. Four (8.3%) evaluate evasion, adversarial manipulation, or concept drift [34], [55], [56], [77]. The counts do not measure explanation quality or attack coverage; they show only that the study reports an explicit mechanism or evaluation. Email-only explanation and prompt-injection studies remain relevant transfer evidence [117], [121], but are not included in these SMS counts.

Device evidence. COPS is the only row reporting both inference time and processing memory on a named handset [42]. SMSPROTECT and the character-level system of Seo et al. demonstrate a handset implementation but do not provide a complete resource profile [56], [81]. No row reports detector performance at an explicit quantized precision or energy per classification. This is a reporting gap, not proof that no unpublished or commercial implementation has made such measurements.

B. WHY REPORTED SCORES DO NOT RANK ARCHITECTURES

Published headline scores cluster in the upper nineties, especially on the 2011 SMS Spam Collection. That pattern is compatible with benchmark saturation, but it does not isolate a cause. The studies differ in duplicate handling, preprocessing, train/test construction, label space, class balance, metric, hyperparameter search, and sometimes the underlying channel. The canonical corpus is known to contain near-duplicate spam messages [86], [87], which creates a leakage risk when related messages cross a random split. Cross-paper results cannot quantify how much of any score is caused by duplication, architecture, or preprocessing. The controlled pilot in Section XII addresses only the split component on two named corpora and shows why that narrower claim is identifiable.

The same caution applies to constrained settings. A federated Bi-LSTM reports 88.78% accuracy [82], but that result does not establish that federation ''costs'' accuracy because there is no controlled comparison against every centralized result shown elsewhere. Similarly, results from a Swahili corpus [28], a tri-class English dataset [49], an evasion-aware collection [55], and a code-switched Filipino–English study [151] answer different questions. Reported score differences should therefore be read within each study's design, not across papers.

The review also cannot rank architectures for code-mixed Indian smishing. Character-level channels are a plausible response to spelling variation and evasion [30], [56]; MuRIL and IndicBERT are plausible transfer baselines [62], [63]; and compact decoder models create a new deployment option [38]–[40]. None of these propositions has been tested on a shared fraud-labeled Indian SMS corpus. A controlled benchmark, rather than further cross-paper comparison, is needed to determine which design works best.

C. INTERPRETING THE ENGLISH CONCENTRATION

The data establish that English dominates the eligibility-filtered comparison subset; they do not establish why authors chose English. One plausible explanation is the low cost and direct comparability of reusing a public benchmark, whereas a Malayalam–English study requires lawful collection, expert fraud annotation, script handling, and adjudication of ambiguous spam/smishing cases. Tooling may reinforce that choice because tokenizers and pretrained representations perform unevenly on Romanized Indic text [14]. These are interpretations supported by the surrounding resource landscape, not measured motivations of study authors.

The practical consequences are clearer. First, high English benchmark scores do not establish transfer to a code-mixed register. Second, private or single-language corpora prevent a common cross-language comparison. Third, evaluations that omit campaign-, sender-, and time-separated splits cannot show resistance to temporal or campaign leakage. Finally, a model that fits in server memory has not thereby been shown to satisfy handset privacy, latency, or energy requirements.

D. ANSWERS TO THE RESEARCH QUESTIONS

RQ1 (method evolution). The literature contains useful methods from rules through decoder language models. Later families add transfer, prompting, augmentation, and explanation, but heterogeneous protocols prevent an architecture ranking across studies.

RQ2 (code-mixed and low-resource evidence). Transliteration resources, Indic encoders, code-mix-specific models,

TABLE 12. Reported Capability Coverage in Selected Direct SMS Systems

Representative system	Generation	Non-English	Code-mixed / Indic	Explain-ability	Robustness evaluated	On-device profiling	Binding limitation
SMSPROTECT [81]	Rule / tree	×	×	✓	×	×	Client–server design; rules require maintenance
Salman et al. [55]	Classical	×	×	×	✓	×	Shallow models fragile under evasion
Ramanujam et al. [105], [145]	Classical	✓	✓[b]	✓	×	×	Spam/ham only; hand-crafted features
Seo et al. [56]	Deep (char-CNN)	Not stated	×	×	✓	✓[a]	Compact character model; no bit-width
COPS [42]	Deep (VAE)	×	×	×	×	✓[e]	English only; no energy or quantization result
Tanbhir et al. [30]	Deep + PLM	✓	×	×	×	×	Single language; small corpus
ExplainableDetector [23]	PLM (RoBERTa)	×	×	✓	×	×	Saturated 2011 English benchmark
Federated PhoBERT [31]	PLM (federated)	✓	×	×	×	×[c]	Training-time privacy only
SOUPS LLM agents [26]	LLM (agentic)	×	×	✓	×	×	Cloud agent loops; latency/cost
KorSmishing Explainer [29]	LLM (PEFT)	✓	×	✓	×	×	Korean only; no edge or robustness evaluation
SpaLLM-Guard [34]	LLM	×	×	×	✓	×	English SMS; large-model serving cost
PhishNet [78]	LLM (ensemble)	×	×	×	×	×	Ensemble serving cost
LoRA multilingual [43]	PEFT (XLM-R)	✓	×	×	×	×	Encoder-only; unquantized; no Indic
Corpuz et al. [151]	Classical	✓	✓[d]	×	×	×	Filipino–English; not Indic; pre-PLM

A check mark indicates that the cited study reports evidence for the column; it is not a quality score. [a]Handset implementation without latency, memory, and energy at the deployed precision. [b]Dravidian languages with spam/ham labels only. [c]Federated *training*; device inference is not profiled. [d]Code-switched Filipino–English, not an Indian language pair. [e]Named-handset model size, processing memory, and inference time, but not energy or deployed numerical precision.

and Malayalam–English language-identification data provide a strong adjacent toolkit [13], [61]–[63]. Direct Indian-language SMS studies remain mostly classical and binary-labeled; no reviewed source supplies the full code-mixed smishing task.

RQ3 (efficiency). Quantization, parameter-efficient adaptation, and mobile profiling have been evaluated separately [33], [37], [41]. COPS supplies a direct handset time-and-memory precedent [42]. Their joint effect on code-mixed, adversarial SMS classification, including energy and deployed precision, remains unmeasured.

RQ4 (datasets and protocols). Evaluation is fragmented by corpus, labels, splits, and metrics. The preliminary validation finds substantial exact duplication in BangalaBarta, a measurable naive-to-grouped split difference on the English tri-class corpus, and extensive source overlap in the proposed UCI transfer test. Credible comparison therefore requires deduplication, provenance-audited group- and time-aware splits, class-specific metrics, uncertainty reporting, and hardware-grounded measurements.

RQ5 (research priorities). Data governance and a fraud-aware code-mixed corpus must precede architecture comparison. The pilot validates simple controls and leakage checks, but does not fill the code-mixed data or Indic-model gaps.

TABLE 13. Cross-Study Deployment Evidence. N/R Means Not Reported; Results Are Not Directly Comparable Across Datasets, Splits, Label Spaces, or Metrics

System	Reported result	Parameters / model size	Memory	Latency	Energy	Deployed precision	Explainability	What the row establishes
SMSPROTECT [81]	Mobile client–server detector	N/R	N/R	N/R	N/R	N/R	Induced rules	Android interface with remote detection service; not fully local inference
Seo et al. [56]	Robustness gain; handset implementation	N/R	N/R	N/R	N/R	N/R	N/R	Robustness evidence; resource profile absent
COPS [42]	98.15% accuracy	3.46 MB	∼12 MB	∼0.08 ms/character	N/R	N/R	N/R	Named-handset time-and-memory profile
ExplainableDetector [23]	99.84% accuracy	N/R	N/R	N/R	N/R	N/R	LIME + attribution	Strong explanation evidence on a saturated English benchmark
KorSmishing Explainer [29]	15% gain over GPT-4	<5B parameters	N/R	N/R	N/R	N/R	Generated rationales evaluated	Non-English classification and explanation; no edge evidence
SpaLLM-Guard [34]	98.61% accuracy	N/R	N/R	N/R	N/R	N/R	N/R	SMS LLM robustness evidence; no deployment profile
LoRA multilingual smishing [43]	Up to 97% F1	XLM-R + LoRA; count N/R	N/R	N/R	N/R	Unquantized	N/R	Low-sample multilingual PEFT; no Indic or device result
Federated Bi-LSTM [82]	88.78% accuracy	N/R	N/R	N/R	N/R	N/R	N/R	Privacy-preserving training; device cost not reported

The empty cells are a result, not missing extraction: no row in the eligibility-filtered comparison set smishing detector reports accuracy or F1 together with parameters/model size, resident memory, latency, energy, deployed numerical precision, and an explanation evaluation. Cross-paper averaging is therefore not statistically defensible.

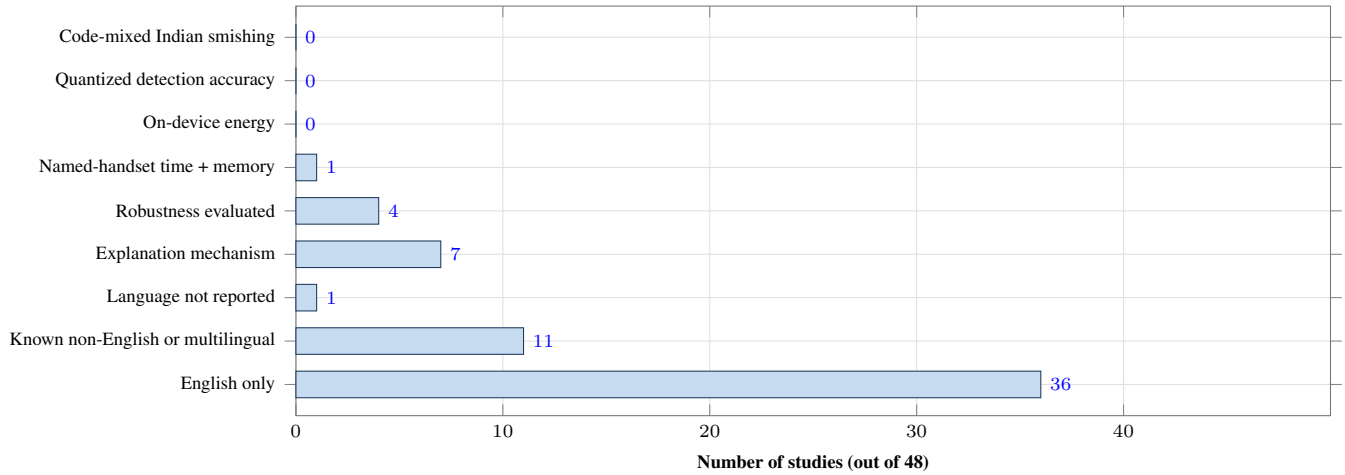

FIGURE 9. Evidence coverage in the 48 tabled direct SMS studies. Language categories partition the set; capabilities may overlap. No row reports code-mixed Indian smishing, accuracy at a stated quantized precision, or on-device energy.

Once that dataset and protocol are fixed, Indic encoders, compact decoder models, compression, robustness, explanations, and device profiling can be compared under the same conditions. Section XIV states that dependency order.

XIV. RESEARCH GAPS AND A DEPENDENCY-ORDERED AGENDA

The evidence base contains many of the components needed for code-mixed Indian-language smishing detection, but it does not evaluate them together under one controlled target-task protocol. Section XII verifies the simple-control, duplicate-grouping, and provenance-audit machinery on available English and Bangla proxies; it does not supply the missing code-mixed Indian corpus or an Indic-model comparison. Table 14 therefore states eight remaining gaps as missing artifacts or tests. The order matters: data governance and label validity must be settled before a model comparison, and model comparison must precede claims about compression or deployment.

A. DATA AND GOVERNANCE FIRST

The first artifact should be a governance plan, not a model. SMS can contain names, account details, transaction information, authentication codes, health information, and private conversations. Collection must therefore establish a lawful basis, consent or an approved alternative, retention limits, access control, redaction of personally identifiable information, and a clear license. The sampling plan should document region, dialect, script, socioeconomic coverage, device source, and collection period so that omissions are visible rather than hidden in an aggregate count.

The label space also requires adjudication. Bulk promotion can be unlawful or unwanted without being fraudulent, while a smishing message may resemble a legitimate bank or de-

TABLE 14. Evidence Gaps, Required Artifacts, and Dependencies

Gap	Evidence boundary	Required artifact or test	Dependency
G1: Governance and provenance	Existing Indic SMS resources do not supply a common account of lawful collection, consent, personally identifiable information removal, dialect/script coverage, or synthetic-data provenance	Governance plan; data statement; annotator protocol; authentic-only held-out test set; campaign/source leakage controls	Before collection
G2: Fraud-aware code-mixed data	Reviewed Indian SMS corpora are spam/ham labeled [58], [148], [149]; tri-class data exist in English [49] and single-script Bangla [169], not the target code-mixed Indian setting	Expert-adjudicated ham/spam/smishing corpus with native and Roman scripts, source/time metadata, and clear release conditions	G1
G3: Comparable evaluation	Metrics, splits, and duplicate handling differ across studies [86], [87]; the proxy pilot quantifies split sensitivity but lacks campaign/time identifiers	Preregistered deduplication; campaign-, sender-, and time-aware splits; per-class metrics, calibration, confidence intervals, and multiple seeds	G2
G4: Indic model evidence	MuRIL and IndicBERT are not evaluated on fraud-labeled Indian SMS in the reviewed evidence [62], [63]	Native-, Roman-, and transliteration-controlled encoder baselines; classical and character-level controls	G2–G3
G5: Precision-aware adaptation	General quantization results do not establish detector behavior [37], [41]	Separate FP16/BF16 LoRA, NF4 QLoRA training, and AWQ/GPTQ inference conditions; per-class and per-script deltas	G3–G4
G6: Device measurement	Named-handset time and memory are rare and energy is not reported in the coded subset [42]	Cold/warm batch-one latency, resident and peak memory, energy per message, hardware/OS/backend, and burst-load tests at stated precision	G5
G7: Robustness at shipped precision	Evasion affects full-precision detectors [55], [56]; compression interactions remain unmeasured	Identical evasion suites across full and compressed variants; prompt-injection tests for instruction-following components	G5–G6
G8: Explanations and private adaptation	Code-mixed Indic rationale quality and federated compressed-model adaptation were not identified	Native-speaker assessment of faithfulness, usefulness, and localization; federated tests covering poisoning, gradient leakage, secure aggregation, and differential privacy	G2–G7

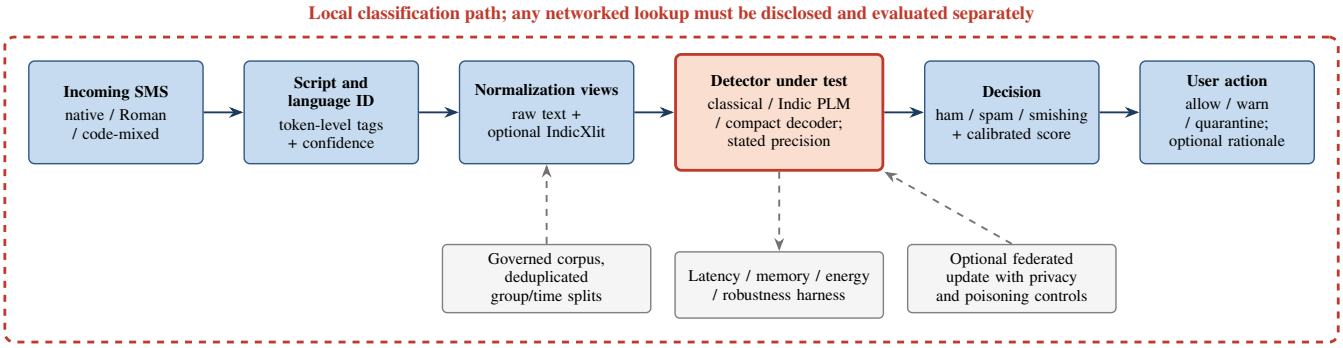

FIGURE 10. Dependency-aware reference architecture for evaluating code-mixed smishing detection. Language identification precedes optional transliteration; profiling and robustness tests apply to the detector at its deployed precision.

livery notification. The annotation guide should define ham, spam, and smishing through intent and requested action; provide an uncertain category during annotation; use at least two trained annotators plus adjudication; and report agreement. Persuasion labels can be added as a secondary layer [25], [52], but they should not substitute for fraud adjudication.

Synthetic messages can supplement scarce cases but must be tracked by generator, prompt, model version, and seed. They should never enter the authentic-only test set. Splits should group messages by campaign, sender, template, URL domain, and near-duplicate cluster before applying a time boundary; otherwise, paraphrases of the same campaign can leak across training and test data.

B. A MINIMUM CORE BENCHMARK

Once the corpus is frozen, the following sequence produces interpretable evidence.

S1. Preregister the evaluation. Specify deduplication, group- and time-aware splits, macro-F1, per-class precision and recall, false positives per 1,000 legitimate messages, calibration, confidence intervals, random seeds, and at least one cross-corpus test.

S2. Establish simple controls. Evaluate a lexical classical model and a character-level model before adding larger architectures. These controls reveal whether complexity adds value on the same data.

S3. Test Indic encoders. Compare MuRIL and IndicBERT [62], [63] on raw native-script, raw Romanized, and optional IndicXlit-normalized views [61]. Script and language identification should precede selective transliteration, as shown in Fig. 10.

S4. Separate adaptation from inference compression. Use FP16 or BF16 LoRA as the full-precision adapted baseline; evaluate NF4 QLoRA as a quantized-training condition [37]; and evaluate AWQ or GPTQ as post-training inference conditions [35], [36]. Report per-class and per-script changes rather than a single average.

S5. Measure the shipped artifact. On named hardware, report cold and warm batch-one classification latency, peak and resident memory, energy per message, model size, precision, runtime/backend, processor path, operating-system version, and message length. Test ordinary single-message arrival and a burst condition; thermal behavior from generative workloads should not be assumed to apply without measurement [33], [73].

S6. Attack the same artifact. Apply the same homoglyph, zero-width, spacing, URL, transliteration, and paraphrase tests to full-precision and compressed variants [55], [56]. Instruction-following components also require prompt-injection tests [121].

S7. Evaluate explanations and private learning separately. Native speakers should assess rationale faithfulness, usefulness, localization, and code-mixed naturalness. Federated adaptation should report non-independent and non-identically distributed (non-IID) behavior, gradient leakage, poisoning resistance, secure aggregation, and any differential-privacy budget [31].

Steps S1–S6 form the minimum core benchmark. S7 evaluates user-facing explanation and private adaptation after detector validity and deployment cost are known. The preliminary validation instantiates S2 and the cross-corpus portion of S1, but the available corpora lack campaign and time identifiers and do not support the held-language-out rotation. S3 remains untested on a fraud-labeled code-mixed Indian corpus. Model candidates should be selected at preregistration using reproducible criteria—license, parameter count, mobile-runtime support, context requirements, and language coverage—rather than a claim that one product family is permanently current. The central experiment is the joint behavior of language, robustness, precision, and device cost under one protocol.

XV. CONCLUSION

This critical review connects direct SMS threat detection with code-mixed Indic processing and resource-efficient language-model inference. The quantitative map covers 48 direct SMS studies in the manuscript's comparison tables. Of these, 36 studies are English-only, eleven include a known non-English or multilingual evaluation, seven report an explanation mechanism, four evaluate robustness, and one reports both time and processing memory on a named handset. The reviewed evidence contains neither a public code-mixed Indian-language corpus that separates ham, bulk spam, and smishing nor a study that jointly reports code-mixed Indic performance, numerical precision, robustness, latency, memory, and energy.

The preliminary validation converts two of those recommendations into evidence. Exact deduplication and conflict quarantine leave 5,797 of 5,971 English rows and 1,204 of 2,772 Bangla rows. Near-duplicate grouping lowers mean English macro-F1 relative to naive splitting by 0.031 for the word SVM and 0.027 for the character SVM. A separate audit shows that only 408 of 5,159 unique UCI messages remain after removing exact and near overlap with the English source, so an unfiltered transfer score would be contaminated. These results validate the evaluation machinery, not the proposed code-mixed Indian detector.

Reported cross-paper scores still do not support an architecture ranking because the underlying data, labels, splits, and metrics differ. The next useful contribution is therefore the full controlled benchmark: governed authentic data in native and Roman scripts; leakage-resistant splits; simple, Indic-encoder, and compact-decoder baselines; separate full-precision, quantized-training, and post-training-compression conditions; adversarial testing; and measurements on named devices. These components have been studied separately. Their behavior when evaluated together remains an open empirical question.

DATA AND MATERIALS AVAILABILITY

The supplementary material contains the search framework, canonical query definitions, eligibility rules, boundary-claim audit, the complete reference catalogue, and a study-level comparison ledger with explicit core-eligibility decisions. The accompanying empirical package adds repository archive hashes, duplicate and conflict audits, text-hash split indices, per-seed predictions, result summaries, and executable scripts; licensed source messages are obtained from their cited repositories and are not redistributed. Because historical database exports and record-level screening decisions were not preserved, the review material supports reproducibility of the reported coding but not reconstruction of a PRISMA record flow.

REFERENCES

[1] M. M. A. Pritom *et al.*, "Short Message Service (SMS) phishing attacks and defenses: A systematic review," arXiv:2604.11429, 2026.

[2] S. Mishra and D. Soni, "Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis," *Future Gener. Comput. Syst.*, vol. 108, pp. 803–815, 2020, doi: 10.1016/j.future.2020.03.021.

[3] U.S. Federal Trade Commission, "New FTC data show top text message scams of 2024 as overall losses to text scams hit $470 million," Apr. 16, 2025. [Online]. Available: https://www.ftc.gov/news-events/news/press

-releases/2025/04/new-ftc-data-show-top-text-message-scams-2024-overall-losses-text-scams-hit-470-million. Accessed: Jul. 27, 2026.

[4] D. Timko, D. H. Castillo, and M. L. Rahman, "Understanding influences on SMS phishing detection: User behavior, demographics, and message attributes," in *Proc. NDSS Workshop Usable Security (USEC)*, 2025, doi: 10.14722/usec.2025.23027.

[5] A. Nahapetyan *et al.*, "On SMS phishing tactics and infrastructure," in *IEEE Symp. Security and Privacy (S&P)*, 2024, doi: 10.1109/SP54263.2024.00169.

[6] M. Liu, Y. Zhang, B. Liu, Z. Li, H. Duan, and D. Sun, "Detecting and characterizing SMS spearphishing attacks," in *ACSAC*, 2021, pp. 930–943, doi: 10.1145/3485832.3488012.

[7] D. Timko and M. L. Rahman, "Commercial anti-smishing tools and their comparative effectiveness against modern threats," in *ACM WiSec*, 2023, pp. 1–12, doi: 10.1145/3558482.3590173.

[8] A. M. Shibli, M. M. A. Pritom, and M. Gupta, "AbuseGPT: Abuse of generative AI chatbots to create smishing campaigns," in *IEEE ISDFS*, 2024, pp. 1–6.

[9] D. Sivaneswaran, C. T.E.R. Hewage, H.M.K.K.M.B. Herath, R. S. Rathore, V. K. Singh, and W. Jiang, "A systematic literature review of large language models in phishing attack generation and detection," *Array (Elsevier)*, 2026, doi: 10.1016/j.array.2026.100775.

[10] Government of India, Press Information Bureau, Ministry of Home Affairs, "Cyber crime reporting and investigation," Feb. 11, 2026. [Online]. Available: https://pib.gov.in/PressReleasePage.aspx?PRID=2226441. Accessed: Jul. 27, 2026.

[11] Telecom Regulatory Authority of India, "Direction regarding implementation of UCC Detect System under the Telecom Commercial Communications Customer Preference Regulations, 2018," Jun. 13, 2023. [Online]. Available: https://www.trai.gov.in/release-publication/directions?page=1. Accessed: Jul. 27, 2026.

[12] B. R. Chakravarthi *et al.*, "DravidianCodeMix: Sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text," *Lang. Resour. Eval.*, vol. 56, no. 3, pp. 765–806, 2022, doi: 10.1007/s10579-022-09583-7.

[13] S. Thara and P. Poornachandran, "Corpus creation and transformer based language identification for code-mixed Indian language," *IEEE Access*, vol. 9, pp. 118837–118850, 2021, doi: 10.1109/access.2021.3104106.

[14] J. Krishnan, A. Anastasopoulos, H. Purohit, and H. Rangwala, "Cross-lingual text classification of transliterated Hindi and Malayalam," in *IEEE Big Data*, 2022; arXiv:2108.13620.

[15] T. A. Almeida, J. M. Gomez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *ACM DOCENG*, 2011, doi: 10.1145/2034691.2034742.

[16] D. Goel and A. K. Jain, "Smishing-Classifier: A novel framework for detection of smishing attack in mobile environment," in *NGCT* (Springer), 2018, pp. 502–512, doi: 10.1007/978-981-10-8660-1_38.

[17] J. W. Joo, S. Y. Moon, S. Singh, and J. H. Park, "S-Detector: An enhanced security model for detecting smishing attack for mobile computing," *Telecommun. Syst.*, vol. 66, no. 1, pp. 29–38, 2017, doi: 10.1007/s11235-016-0269-9.

[18] J. M. Gomez Hidalgo, G. Cajigas Bringas, E. Puertas Sanz, and F. Carrero Garcia, "Content based SMS spam filtering," in *Proc. ACM Symp. Document Engineering (DocEng)*, 2006, pp. 107–114, doi: 10.1145/1166160.1166191.

[19] M. Soysaldi Sahin, D. O. Sahin, and A. A. F. Salah, "Revisiting SMS spam detection: The impact of feature representation on classical machine learning models," *Electronics*, vol. 15, no. 4, art. 894, MDPI, 2026, doi: 10.3390/electronics15040894.

[20] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep learning to filter SMS spam," *Future Gener. Comput. Syst.*, vol. 102, pp. 524–533, 2020, doi: 10.1016/j.future.2019.09.001.

[21] A. Ghourabi, M. A. Mahmood, and Q. M. Alzubi, "A hybrid CNN-LSTM model for SMS spam detection in Arabic and English messages," *Future Internet*, vol. 12, no. 9, art. 156, 2020, doi: 10.3390/fi12090156.

[22] A. Ghourabi, "SM-Detector: A security model based on BERT to detect SMiShing messages in mobile environments," *Concurrency Comput.: Pract. Exp.*, vol. 33, no. 24, art. e6452, 2021, doi: 10.1002/cpe.6452.

[23] M. A. Uddin, M. N. Islam, L. Maglaras, H. Janicke, and I. H. Sarker, "ExplainableDetector: Exploring transformer-based language modeling approach for SMS spam detection with explainability analysis," *Digit. Commun. Netw.*, vol. 11, no. 5, pp. 1504–1518, 2025, doi: 10.1016/j.dcan.2025.07.008; arXiv:2405.08026.

[24] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and detecting phishing emails using large language models," *IEEE Access*, vol. 12, pp. 42131–42146, 2024, doi: 10.1109/ACCESS.2024.3375882; arXiv:2308.12287.

[25] H. S. Shim, H. Park, K. Lee, J. Park, and S. Kang, "Data augmentation for smishing detection: A theory-based prompt engineering approach," in *Companion Proc. ACM Web Conf.*, 2024, pp. 1327–1328, doi: 10.1145/3589335.3651903.

[26] Y. Wang *et al.*, "Can you walk me through it? Explainable SMS phishing detection using LLM-based agents," in *USENIX SOUPS*, 2025, pp. 37–56.

[27] S. M. Sanjari, S. Roberts, and M. M. A. Pritom, "Poster: A few-shot learning method for SMS phishing detection and explanation using SLMs," in *IEEE S&P* (poster), 2025.

[28] I. S. Mambina, J. D. Ndibwile, and K. F. Michael, "Classifying Swahili smishing attacks for mobile money users: A machine-learning approach," *IEEE Access*, vol. 10, pp. 83061–83074, 2022, doi: 10.1109/ACCESS.2022.3196464.

[29] Y. Lee and D. Han, "KorSmishing Explainer: A Korean-centric LLM-based framework for smishing detection and explanation generation," in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing: Industry Track*, 2024, pp. 642–656, doi: 10.18653/v1/2024.emnlp-industry.47.

[30] G. Tanbhir, M. F. Shahriyar, K. Shahed, A. M. R. Chy, and M. A. Adnan, "Hybrid machine learning model for detecting Bangla smishing text using BERT and character-level CNN," in *IEEE ICECE*, 2024, pp. 57–62, doi: 10.1109/ICECE64886.2024.11024872.

[31] H. Q. Anh, P. T. Anh, P. S. Nguyen, and P. D. Hung, "Federated learning for Vietnamese SMS spam detection using pre-trained PhoBERT," in *IDEAL*, LNCS vol. 15346, Springer, 2025, pp. 254–264, doi: 10.1007/978-3-031-77731-8_24.

[32] M. ElZemity *et al.*, "Agentic knowledge distillation: Autonomous training of small language models for SMS threat detection," arXiv:2602.10869, 2026.

[33] R. Murthy *et al.*, "MobileAIBench: Benchmarking LLMs and LMMs for on-device use cases," arXiv:2406.10290, 2024.

[34] M. Salman, M. Ikram, N. Basta, and M. A. Kaafar, "SpaLLM-Guard: Pairing SMS spam detection using open-source and commercial LLMs," arXiv:2501.04985, 2025.

[35] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *ICLR*, 2023.

[36] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," in *MLSys*, 2024.

[37] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *NeurIPS*, 2023; arXiv:2305.14314.

[38] M. Abdin *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," arXiv:2404.14219, 2024.

[39] Gemma Team, "Gemma 2: Improving open language models at a practical size," arXiv:2408.00118, 2024.

[40] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024, doi: 10.48550/arXiv.2412.15115.

[41] J. Lee, S. Park, J. Kwon, J. Oh, and Y. Kwon, "Exploring the trade-offs: Quantization methods, task difficulty, and model size in large language models from edge to giant," arXiv:2409.11055 (v6, 2025).

[42] H. B. S. S. Harichandana, S. Kumar, M. B. Ujjinakoppa, and B. R. K. Raja, "COPS: A compact on-device pipeline for real-time smishing detection," in *Proc. IEEE Consumer Communications & Networking Conf. (CCNC)*, 2024, pp. 172–179, doi: 10.1109/CCNC51664.2024.10454900.

[43] N. Krawczyk, B. Probierz, and J. Kozak, "Fine-tuning of multilingual language models for low-resource smishing detection using LoRA," in *Intelligent Information and Database Systems (ACIIDS)*, LNAI vol. 16530, Springer, 2026, doi: 10.1007/978-981-92-0071-9_8.

[44] M. Rahaman, N. Cajes, B. B. Gupta, K. T. Chui, and N. Nedjah, "Defending against smishing attacks: State-of-the-art techniques, challenges, limitations, and future directions," *Comput. Netw.*, 2025, doi: 10.1016/j.comnet.2025.111758.

[45] M. R. A. Saidat, S. Y. Yerima, and K. Shaalan, "Advancements of SMS spam detection: A comprehensive survey of NLP and ML techniques," *Procedia Comput. Sci.*, vol. 244, pp. 248–259, 2024, doi: 10.1016/j.procs.2024.10.198.

[46] A. Kulkarni, V. Balachandran, and T. Das, "Phishing webpage detection: Unveiling the threat landscape and investigating detection techniques," *IEEE Commun. Surveys Tuts.*, 2024, doi: 10.1109/COMST.2024.3441752.
[47] D. Goel and A. K. Jain, "Mobile phishing attacks and defence mechanisms: State of art and open research challenges," *Comput. Secur.*, vol. 73, pp. 519–544, 2017, doi: 10.1016/j.cose.2017.12.006.
[48] C. V. Nguyen *et al.*, "A survey of small language models," arXiv:2410.20011, 2024.
[49] S. Mishra and D. Soni, "SMS phishing dataset for machine learning and pattern recognition," in *SoCPaR* (Springer), 2022, pp. 597–604, doi: 10.1007/978-3-031-27524-1_57.
[50] S. Mishra and D. Soni, "SMS phishing dataset for machine learning and pattern recognition," Mendeley Data, ver. 1, 2022, doi: 10.17632/f45bkkt8pr.1.
[51] M. F. Islam and M. L. Munoz, "Deep learning approaches for multi-class classification of phishing text messages," *J. Cybersecur. Priv.*, vol. 5, no. 4, art. 102, MDPI, 2025, doi: 10.3390/jcp5040102.
[52] R. B. Cialdini, *Influence: Science and Practice*, 4th ed. Boston, MA, USA: Allyn and Bacon, 2001.
[53] C. Zhong, W. Xie, and N. El Shamy, "Demystifying persuasion techniques in smishing with explainable AI," in *Proc. AMCIS TREO Talks*, 2024.
[54] S. Mishra and D. Soni, "DSmishSMS: A system to detect smishing SMS," *Neural Comput. Appl.*, vol. 35, no. 7, pp. 4975–4992, 2023, doi: 10.1007/s00521-021-06305-y.
[55] M. Salman, M. Ikram, and M. A. Kaafar, "Investigating evasive techniques in SMS spam filtering: A comparative analysis of machine learning models," *IEEE Access*, vol. 12, pp. 24306–24324, 2024, doi: 10.1109/ACCESS.2024.3364671.
[56] J. W. Seo *et al.*, "On-device smishing classifier resistant to text evasion attack," *IEEE Access*, vol. 12, pp. 4762–4779, 2024, doi: 10.1109/ACCESS.2024.3349577.
[57] M. Tabassum, C. Faklaris, and H. R. Lipford, "What drives SMiShing susceptibility? A U.S. interview study of how and why mobile phone users judge text messages to be real or fake," in *USENIX SOUPS*, 2024, pp. 393–411.
[58] R. Elangovan and A. M. Abirami, "Spam SMS in Dravidian languages (dataset)," IEEE DataPort, 2023, doi: 10.21227/dcym-pd69.
[59] V. Srivastava and M. Singh, "IIT Gandhinagar at SemEval-2020 Task 9: Code-mixed sentiment classification using candidate sentence generation and selection," in *Proc. SemEval-2020*; arXiv:2006.14465.
[60] B. Roark *et al.*, "Processing South Asian languages written in the Latin script: The Dakshina dataset," in *Proc. LREC*, 2020, pp. 2413–2423; arXiv:2007.01176.
[61] Y. Madhani *et al.*, "Aksharantar: Open Indic-language transliteration datasets and models for the next billion users (IndicXlit)," in *Findings of EMNLP*, 2023, pp. 40–57; arXiv:2205.03018.
[62] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730 (Google Research), 2021.
[63] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages (IndicBERT)," in *Findings of EMNLP*, 2020.
[64] A. Vaswani *et al.*, "Attention is all you need," in *NeurIPS*, vol. 30, 2017, pp. 5998–6008.
[65] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019.
[66] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale (XLM-R)," in *ACL*, 2020.
[67] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," in *EMC2 Workshop @ NeurIPS*, 2019; arXiv:1910.01108.
[68] T. B. Brown *et al.*, "Language models are few-shot learners (GPT-3)," in *NeurIPS*, vol. 33, 2020, pp. 1877–1901.
[69] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv (NIPS 2014 DL Workshop), 2015.
[70] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for LLMs," in *ICML*, 2023.
[71] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *NeurIPS*, 2022.
[72] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *ICLR*, 2022; arXiv:2106.09685.
[73] P. Tummalapalli, S. Arayakandy, R. Pal, and K. Kundan, "LLM inference at the edge: Mobile, NPU, and GPU performance efficiency trade-offs under sustained load," arXiv:2603.23640, 2026.
[74] G. Sonowal and K. S. Kuppusamy, "SMIDCA: An anti-smishing model with machine learning approach," *Comput. J.*, vol. 61, no. 8, pp. 1143–1157, 2018, doi: 10.1093/comjnl/bxy039.
[75] M. S. Mahmud, M. A. Rahman, M. S. Hossain, and M. M. A. Pritom, "Enhancing cybersecurity: Hybrid deep learning approaches to smishing attack detection (CNN-Bi-GRU)," *Systems*, vol. 12, no. 11, art. 490, MDPI, 2024, doi: 10.3390/systems12110490.
[76] F. Ji and D. Kim, "How can we effectively use LLMs for phishing detection?," arXiv:2511.09606, 2025.
[77] S. Hur *et al.*, "Are large language models effective for detecting spam messages?," *Int. J. Inf. Secur.*, vol. 25, art. 127, 2026, doi: 10.1007/s10207-026-01283-5.
[78] A. Alhuzali, Q. Al-Qahtani, A. Niyazi, L. Alshehri, and F. Alharbi, "PhishNet: A real-time, scalable ensemble framework for smishing attack detection using transformers and LLMs," *Comput. Mater. Continua*, vol. 86, no. 1, 2026, doi: 10.32604/cmc.2025.069491.
[79] A. K. Jain, B. B. Gupta, K. Kaur, P. Bhutani, W. Alhalabi, and A. Almomani, "A content and URL analysis-based efficient approach to detect smishing SMS in intelligent systems," *Int. J. Intell. Syst.*, vol. 37, no. 12, pp. 11117–11141, 2022.
[80] K. Yadav, P. Kumaraguru, A. Goyal, A. Gupta, and V. Naik, "SMSAssassin: Crowdsourcing driven mobile-based system for SMS spam filtering," in *Proc. ACM HotMobile*, 2011, pp. 1–6, doi: 10.1145/2184489.2184491.
[81] O. N. Akande *et al.*, "SMSPROTECT: An automatic smishing detection mobile application," *ICT Express*, vol. 9, no. 2, pp. 168–176, 2023, doi: 10.1016/j.icte.2022.05.009.
[82] M. A. Remmide, F. Boumahdi, B. Ilhem, and N. Boustia, "A privacy-preserving approach for detecting smishing attacks using federated deep learning," *Int. J. Inf. Technol.*, 2024, doi: 10.1007/s41870-024-02144-x.
[83] J. Sidhpura, P. Shah, R. Veerkhare, and A. Godbole, "FedSpam: Privacy preserving SMS spam prediction," in *Neural Information Processing (ICONIP 2022)*, CCIS vol. 1793, Springer, 2023, doi: 10.1007/978-981-99-1645-0_5.
[84] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is BERT really robust? A strong baseline for natural language attack on text classification and entailment (TextFooler)," in *AAAI*, 2020; arXiv:1907.11932.
[85] T. A. Almeida and J. M. Gomez Hidalgo, "SMS Spam Collection v.1 (UCI repository entry)," UCI Machine Learning Repository, 2012, doi: 10.24432/C5CC84.
[86] J. M. Gomez Hidalgo, T. A. Almeida, and A. Yamakami, "On the validity of a new SMS spam collection," in *IEEE ICMLA*, 2012.
[87] T. A. Almeida, J. M. Gomez Hidalgo, and T. P. Silva, "Towards SMS spam filtering: Results under a new dataset," *Int. J. Inf. Secur. Sci.*, vol. 2, no. 1, 2013.
[88] A. K. Jain and B. B. Gupta, "Rule-based framework for detection of smishing messages in mobile environment," *Procedia Comput. Sci.*, vol. 125, pp. 617–623, 2018, doi: 10.1016/j.procs.2017.12.079.
[89] S. Mishra and D. Soni, "SMS phishing and mitigation approaches," in *IC3*, Noida, 2019, pp. 1–5, doi: 10.1109/IC3.2019.8844920.
[90] S. Mishra and D. Soni, "A content-based approach for detecting smishing in mobile environment," SSRN, pp. 986–993, 2019, doi: 10.2139/ssrn.3356256.
[91] A. Alzahrani and D. B. Rawat, "Comparative study of machine learning algorithms for SMS spam detection," in *IEEE SoutheastCon*, 2019, doi: 10.1109/SoutheastCon42311.2019.9020530.
[92] H. Xu, M. A. Qadir, and R. Sadiq, "Malicious SMS detection using ensemble learning and SMOTE to improve mobile cybersecurity," *Comput. Secur.*, vol. 154, art. 104443, 2025, doi: 10.1016/j.cose.2025.104443.
[93] B. Birthriya, A. Ahlawat, and A. K. Jain, "Machine learning-based smishing detection using fuzzy logic and TF–IDF feature engineering," *Franklin Open*, vol. 14, art. 100506, 2026, doi: 10.1016/j.fraope.2026.100506.
[94] A. Ghourabi and M. Alohaly, "Enhancing spam message classification and detection using transformer-based embedding and ensemble learning," *Sensors*, vol. 23, no. 8, art. 3861, 2023, doi: 10.3390/s23083861.
[95] A. K. Jain, D. Goel, S. Agarwal, Y. Singh, and G. Bajaj, "Predicting spam messages using back propagation neural network," *Wireless Pers. Commun.*, vol. 110, no. 1, pp. 403–422, 2020, doi: 10.1007/s11277-019-06734-y.
[96] T. A. Almeida, T. P. Silva, I. Santos, and J. M. Gomez Hidalgo, "Text normalization and semantic indexing to enhance instant messaging and

SMS spam filtering," *Knowl.-Based Syst.*, vol. 108, pp. 25–32, 2016, doi: 10.1016/j.knosys.2016.05.001.

[97] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "Machine learning driven smishing detection framework for mobile security," arXiv:2412.09641, 2024.

[98] R. M. Silva, T. A. Almeida, and A. Yamakami, "MDLText: An efficient and lightweight text classifier," *Knowl.-Based Syst.*, vol. 118, pp. 152–164, 2017, doi: 10.1016/j.knosys.2016.11.018.

[99] S. Bosaeed, I. Katib, and R. Mehmood, "A fog-augmented machine learning-based SMS spam detection and classification system," in *IEEE FMEC*, 2020, pp. 325–330.

[100] A. Theodorus, T. K. Prasetyo, R. Hartono, and D. Suhartono, "Short Message Service (SMS) spam filtering using machine learning in Bahasa Indonesia," in *EIConCIT*, 2021, pp. 199–203.

[101] H. El Karhani, R. Al Jamal, Y. Bou Samra, I. H. Elhajj, and A. Kayssi, "Phishing and smishing detection using machine learning," in *IEEE CSR*, 2023.

[102] E. M. El-Alfy and A. A. AlHasan, "Spam filtering framework for multi-modal mobile communication based on dendritic cell algorithm," *Future Gener. Comput. Syst.*, vol. 64, pp. 98–107, 2016.

[103] G. S. Awumee, J. O. Agyemang, S. S. Boakye, and D. Bempong, "SmishShield: A machine learning-based smishing detection system," in *Proc. 6th WIDECOM*, LNDECT 185, Springer, 2024, pp. 205–221, doi: 10.1007/978-3-031-47126-1_14.

[104] S. R. A. Samad, P. Ganesan, J. Rajasekaran, M. Radhakrishnan, H. Ammaippan, and V. Ramamurthy, "SmishGuard: Leveraging machine learning and natural language processing for smishing detection," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 11, pp. 586–593, 2023, doi: 10.14569/IJACSA.2023.0141160.

[105] E. Ramanujam, K. S. Prakash, and A. M. Abirami, "Evaluation of handcrafted features for the classification of spam SMS in Dravidian languages," in *ICDSNE*, LNNS vol. 791 (Springer), 2024, doi: 10.1007/978-981-99-6755-1_1.

[106] S. M. M. Hossain *et al.*, "Spam filtering of mobile SMS using CNN-LSTM based deep learning model," in *Hybrid Intelligent Systems* (Springer), 2022, pp. 106–116.

[107] U. Maqsood, S. U. Rehman, T. Ali, K. Mahmood, T. Alsaedi, and M. Kundi, "An intelligent framework based on deep learning for SMS and e-mail spam detection," *Appl. Comput. Intell. Soft Comput.*, vol. 2023, art. 6648970 (Wiley), 2023, doi: 10.1155/2023/6648970.

[108] S. Mishra and D. Soni, "Implementation of 'Smishing Detector': An efficient model for smishing detection using neural network," *SN Comput. Sci.*, vol. 3, 2022, doi: 10.1007/s42979-022-01078-0.

[109] M. K. Mehmood, H. Arshad, M. Alawida, and A. Mehmood, "Enhancing smishing detection: A deep learning approach for improved accuracy and reduced false positives," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3463871.

[110] M. M. Uddin, M. Yasmin, M. S. H. Khan, M. I. Rahman, and T. Islam, "Detecting Bengali spam SMS using recurrent neural network," *J. Commun.*, vol. 15, no. 4, pp. 325–331, 2020, doi: 10.12720/JCM.15.4.325-331.

[111] A. P. Arpan, R. Ghatak, M. M. Hasan, A. Roy, M. A. Haque, and S. S. Khan, "BiLSTM-based smishing detection for Bangla SMS," in *Proc. Int. Conf. Intelligent Data Analysis and Applications (IDAA)*, Atlantis Press, 2026, pp. 504–515, doi: 10.2991/978-94-6239-664-7_35.

[112] M. R. Al Saidat, S. Y. Yerima, and K. Shaalan, "A hybrid deep learning framework for Arabic smishing detection: Dataset creation, model design, and explainability," *Nat. Lang. Process. J.*, vol. 16, art. 100222, 2026, doi: 10.1016/j.nlp.2026.100222.

[113] G. Lample and A. Conneau, "Cross-lingual language model pretraining," in *NeurIPS*, 2019.

[114] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[115] A. K. Jain, K. Kaur, N. K. Gupta, and A. Khare, "Detecting smishing messages using BERT and advanced NLP techniques," *SN Comput. Sci.*, vol. 6, no. 2, art. 109, 2025, doi: 10.1007/s42979-024-03532-7.

[116] Z. L. Hassan, N. F. M. Sani, M. D. H. Abdullah, and N. Mustapha, "Transformer attention-driven concept extraction for efficient smishing detection," *Artif. Intell. Appl.*, 2026, doi: 10.47852/bonviewAIA62028760.

[117] M. A. Uddin, M. Mahiuddin, and I. H. Sarker, "An explainable transformer-based model for phishing email detection: A large language model approach," arXiv:2402.13871, 2024.

[118] S. Jamal and H. Wimmer, "An improved transformer-based model for detecting phishing, spam, and ham: A large language model approach," arXiv:2311.04913, 2023.

[119] R. Joshi, "L3Cube-HindBERT and DevBERT: Pre-trained BERT transformer models for Devanagari based Hindi and Marathi," arXiv:2211.11418, 2022.

[120] A. Velankar, H. Patil, and R. Joshi, "Mono vs multilingual BERT for hate speech detection and text classification: A case study in Marathi," in *Proc. ANNPR*, Springer, 2022, pp. 121–128; arXiv:2204.08669.

[121] N. Hasan, P. BusiReddyGari, H. Zhao, Y. Ren, J. Xu, and S. Zhang, "Phishing email detection using large language models (LLMPEA)," arXiv:2512.10104, 2025.

[122] T. Koide, N. Fukushi, H. Nakano, and D. Chiba, "ChatSpamDetector: Leveraging large language models for effective phishing email detection," in *EAI SecureComm*, 2024, pp. 297–319, doi: 10.1007/978-3-031-94455-0_14.

[123] T. Koide, H. Nakano, and D. Chiba, "ChatPhishDetector: Detecting phishing sites using large language models," *IEEE Access*, vol. 12, pp. 154381–154400, 2024, doi: 10.1109/ACCESS.2024.3483905.

[124] D. Nahmias, G. Engelberg, D. Klein, and A. Shabtai, "Prompted contextual vectors for spear-phishing detection," arXiv:2402.08309, 2024.

[125] J. Lee, P. Lim, B. Hooi, and D. M. Divakaran, "Multimodal large language models for phishing webpage detection and identification," in *Proc. APWG eCrime*, 2024, pp. 1–13, doi: 10.1109/eCrime66200.2024.00007; arXiv:2408.05941.

[126] S. M. Sanjari, A. M. Shibli, M. Mia, M. Gupta, and M. M. A. Pritom, "SmishViz: Towards a graph-based visualization system for monitoring and characterizing ongoing smishing threats," in *Proc. ACM CODASPY*, 2025, pp. 257–268, doi: 10.1145/3714393.3726499.

[127] P. Patwa *et al.*, "SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets (SentiMix)," in *Proc. SemEval-2020*; arXiv:2008.04277.

[128] B. R. Chakravarthi, R. Priyadharshini, N. Jose, A. Kumar, T. Mandl, and B. Bharathi, "Overview of the HASOC-DravidianCodeMix shared task on offensive language detection in Tamil and Malayalam," in *FIRE Working Notes*, CEUR Vol-3159, 2021.

[129] C. P. Afsal and K. S. Kuppusamy, "Sentence level language classification of Malayalam-English mixed-language text," in *Proc. ICICBDA*, CCIS 2234, Springer, 2024, doi: 10.1007/978-3-031-74682-6_1.

[130] S. Kazhuparambil and A. Kaushik, "Cooking is all about people: Comment classification on cookery channels using BERT and classification models (Malayalam-English mix-code)," Preprints, 2020, doi: 10.20944/preprints202006.0223.v1.

[131] G. Azam *et al.*, "Beyond specialization: Benchmarking LLMs for transliteration of Indian languages," arXiv:2505.19851, 2025.

[132] M. Z. U. Rehman, D. Raghuvanshi, H. Pachar, C. S. Raghaw, and N. Kumar, "Hierarchical attention-enhanced contextual CapsuleNet for multilingual hope speech detection," *Expert Syst. Appl.*, vol. 268, art. 126285, 2025, doi: 10.1016/j.eswa.2024.126285.

[133] S. Doddapaneni *et al.*, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proc. 61st Annu. Meeting ACL (Vol. 1: Long Papers)*, Toronto, 2023, pp. 12402–12426, doi: 10.18653/v1/2023.acl-long.693; arXiv:2212.05409.

[134] M. Pathak and A. Jain, "muBoost: An effective method for solving Indic multilingual text classification problem," arXiv:2206.10280, 2022.

[135] R. Savant, A. Shelke, S. Todmal, S. Kanphade, A. Joshi, and R. Joshi, "Universal cross-lingual text classification," in *Proc. IEEE I2CT*, 2024, doi: 10.1109/I2CT61223.2024.10543381.

[136] R. Nayak and R. Joshi, "L3Cube-HingCorpus and HingBERT: A code mixed Hindi-English dataset and BERT language models," in *Proc. WILDRE-6 Workshop, LREC*, 2022; arXiv:2204.08398.

[137] M. N. Raihan, D. Goswami, and A. Mahmud, "Mixed-Distil-BERT: Code-mixed language modeling for Bangla, English, and Hindi," arXiv:2309.10272, 2023.

[138] G. Takawane, A. Phaltankar, V. Patwardhan, A. Patil, R. Joshi, and M. S. Takalikar, "Language augmentation approach for code-mixed text classification," *Nat. Lang. Process. J. (Elsevier)*, vol. 5, art. 100042, 2023.

[139] D. Amin *et al.*, "Marathi-English code-mixed text generation," arXiv:2309.16202, 2023.

[140] A. Vaidya, T. Prabhakar, D. George, and S. Shah, "Analysis of Indic language capabilities in LLMs," arXiv:2501.13912, 2025.

[141] S. KJ *et al.*, "IndicMMLU-Pro: Benchmarking Indic large language models on multi-task language understanding," arXiv:2501.15747, 2025.

[142] A. Yadav, T. Garg, M. Klemen, M. Ulcar, B. Agarwal, and M. R. Sikonja, "Code-mixed sentiment and hate-speech prediction (LLMs)," arXiv:2405.12929, 2024.
[143] S. Agarwal, S. Kaur, and S. Garhwal, "SMS spam detection for Indian messages," in *Proc. 1st Int. Conf. Next Generation Computing Technologies (NGCT)*, IEEE, 2015, pp. 634–638, doi: 10.1109/NGCT.2015.7375198.
[144] M. Chakraborty, S. Karmakar, and S. Chattaraj, "Machine learning based Indian spam recognition," *TIU Trans. Intell. Comput.*, vol. 3, pp. 10–16, 2019.
[145] K. G. Thirumalai, K. S. Prakash, A. M. Abirami, E. Ramanujam, and S. Sumitra, "XAI-based feature selection for SMS spam classification in Dravidian languages," in *IEEE ICITIIT*, 2024, pp. 1–6, doi: 10.1109/ICITIIT61487.2024.10580065.
[146] E. Ramanujam, A. M. Abirami, K. Sakthiprakash, and S. Sumitra, "Efficient extraction and evaluation of hand-crafted meta-data features for Dravidian spam SMS classification," *Evolving Syst.*, vol. 17, art. 8, 2026, doi: 10.1007/s12530-025-09761-2.
[147] E. Ramanujam and A. M. Abirami, "NCMDFE: Performance evaluation of non-contextual meta-data feature extraction for spam SMS classification in Indian languages," *Int. J. Inf. Technol.*, vol. 17, pp. 4863–4879, 2025, doi: 10.1007/s41870-025-02635-5.
[148] R. Elangovan and A. M. Abirami, "SMSDHL: Spam SMS in Dravidian and Hindi languages," TechRxiv, 2025, doi: 10.36227/techrxiv.173603655.53756505/v1.
[149] S. Ponnekanty, S. Sahoo, S. Puthiyaveettil, R. Elangovan, and A. M. Abirami, "Spam SMS in Hindi language (dataset)," IEEE DataPort, 2024, doi: 10.21227/5y8x-n678.
[150] B. R. Chakravarthi, M. B. Jagadeeshan, V. Palanikumar, and R. Priyadharshini, "Offensive language identification in Dravidian languages using MPNet and CNN," *Int. J. Inf. Manage. Data Insights*, vol. 3, no. 1, art. 100151, 2023, doi: 10.1016/j.jjimei.2022.100151.
[151] J. E. A. Corpuz, S. Q. Diaz, L. B. M. Palafox, M. C. T. Tan, and K. Y. C. Solomon, "Machine learning-based detection of SMS phishing in the Philippine context," in *Proc. Workshop on Computation: Theory and Practice (WCTP)*, Atlantis Highlights in Computer Sciences, Atlantis Press (Springer Nature), 2026, pp. 553–562, doi: 10.2991/978-94-6239-638-8_28.
[152] T. Dettmers *et al.*, "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," in *ICLR*, 2024; arXiv:2306.03078.
[153] K. Behdin *et al.*, "QuantEase: Optimization-based quantization for language models," arXiv:2309.01885, 2023.
[154] J. Chee, Y. Cai, V. Kuleshov, and C. De Sa, "QuIP: 2-bit quantization of large language models with guarantees," in *NeurIPS*, 2023.
[155] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in *NeurIPS*, 2022.
[156] W. Shao *et al.*, "OmniQuant: Omnidirectionally calibrated quantization for LLMs," arXiv 2023 (ICLR 2024).
[157] Z. Liu *et al.*, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in *Proc. ICML*, 2024; arXiv:2402.14905.
[158] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," arXiv:2401.02385, 2024.
[159] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.
[160] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, "MELTing point: Mobile evaluation of language transformers," in *Proc. 30th ACM MobiCom*, 2024, pp. 890–907, doi: 10.1145/3636534.3690668.
[161] H. Chen, C. Tian, Z. He, B. Yu, Y. Liu, and J. Cao, "Inference performance evaluation for LLMs on edge devices with a novel benchmarking framework and metric," arXiv:2508.11269, 2025.
[162] Z. Nezami, M. Hafeez, K. Djemame, and S. A. R. Zaidi, "Generative AI on the edge: Architecture and performance evaluation," arXiv:2411.17712, 2024.
[163] Z. Zhang, P. Dash, Y. C. Hu, Q. Xu, J. Li, and H. Guan, "Dissecting the impact of mobile DVFS governors on LLM inference performance and energy efficiency," arXiv:2507.02135, 2025.
[164] Y. Huang, H. Hu, and C. Chen, "Robustness of on-device models: Adversarial attack to deep learning models on Android apps," arXiv:2101.04401, 2021.
[165] Y. Huang and C. Chen, "Smart app attack: Hacking deep learning models in Android apps," arXiv:2204.11075, 2022.
[166] T. Chen and M.-Y. Kan, "Creating a live, public short message service corpus: The NUS SMS corpus," *Lang. Resour. Eval.*, vol. 47, no. 2, pp. 299–335, 2013, doi: 10.1007/s10579-012-9197-9.
[167] D. Timko and M. L. Rahman, "Smishing Dataset I: Phishing SMS dataset from smishtank.com," in *Proc. 14th ACM CODASPY*, 2024, pp. 289–294, doi: 10.1145/3626232.3653282.
[168] A. Martinez-Mendoza, E. Fidalgo, E. Alegre, and L. Fernandez-Robles, "Building a multi-class Short Message Service dataset for smishing detection using agglomerative clustering and dataset fusion," *Eng. Appl. Artif. Intell.*, vol. 163, art. 112864 (Elsevier), 2026, doi: 10.1016/j.engappai.2025.112864.
[169] M. Shahriyar and M. Tanbhir, "BangalaBarta: A spam/smishing SMS dataset Bangla," Mendeley Data, ver. 3, 2026, doi: 10.17632/jfkfbw3gzh.3.

SIDHARDH S is currently pursuing the integrated M.Tech. degree in software engineering with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. His research interests include natural language processing, large language models, resource-efficient and on-device machine learning, and SMS-based threat (smishing) detection for low-resource, code-mixed Indian languages.

ADVAITH KAMATH is currently pursuing the integrated M.Tech. degree in software engineering with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. His research interests include machine learning, natural language processing, and cybersecurity.

JOEL G. BERT is currently pursuing the integrated M.Tech. degree in software engineering with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. His research interests include machine learning, natural language processing, and applied deep learning.

T. AMARSAINADH is currently pursuing the integrated M.Tech. degree in software engineering with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. His research interests include machine learning, data science, and text classification.

BILEESH P. BABU received the B.Tech. degree in computer science from TKM College of Engineering, Kollam, India, in 2012, the M.Tech. degree in computer science and engineering from RV College of Engineering, Bengaluru, India, in 2014, and the Ph.D. degree in computer science and engineering from the Vellore Institute of Technology, Vellore, India, in 2025. He is currently an Assistant Professor (Senior Grade 1) with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. His research interests include computer vision, machine learning, and deep learning.

GOKUL YENDURI received the Ph.D. degree from the Vellore Institute of Technology, Vellore, India. He is currently an Assistant Professor (Senior Grade 1) with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Amaravati, India. He has authored more than 50 articles. His research interests include federated learning, blockchain, explainable artificial intelligence, the metaverse, inclusive education, and healthcare informatics. His ORCID is 0000-0001-9146-8378.